J.H. CERILLES STATE COLLEGE
School Of Computing Studies
Mati, San Miguel, Zamboanga del Sur
Main Campus

Floodline Alert: Arduino-Based Flood Warning with SMS Notification

A CAPSTONE PROJECT

Presented to the Faculty of

SCHOOL OF COMPUTING STUDIES

In Partial Fulfillment

Of the Requirements for the degree of

BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY

By:

Rondrique, Kent Peter M.

Bornea, Queennelyn A.

Navales, Sharlene Nicole O.

2026

APPROVAL SHEET

The Capstone Project entitled **Floodline Alert: Arduino-Based Flood Warning with SMS Notification** prepared and submitted by, Kent Peter M. Rondrique, Queennelyn A. Bornea, and Sharlene Nicole O. Navales has been examined and is recommend for approval and acceptance.

RECOMMENDED:

EDILBERTA G. NAPARAN, MSIT
Adviser

APPROVED by the IT Project Defense Panel with a grade of _____.

MARK KENIDE B. BASTASA, MLIS
Chairperson

AMOR DIANNE A. CARILLO, MLIS
Panel Member

DAYAHAN A. BUGAO, MLIS
Panel Member

ACCEPTED and APPROVED in partial fulfillment of the requirements in Bachelor of Science and Information Technology.

MARIBEL M. ESTIOCO, MST-CS
BSIT Program Chair, Main Campus

Date: _____

ACKNOWLEDGEMENT

The researchers wish to express their heartfelt gratitude to the following individuals who played essential roles in completing this study.

To their IT capstone adviser, Ma'am Edilberta G. Naparan, for her unwavering support and wise advice, particularly her exhortation to persevere with diligence and unwavering trust in their abilities, which fortified their resolve throughout the endeavor.

To their IT capstone instructor, Mrs. Maribel M. Estioco, for her unwavering patience, innovative ideas, expert guidance, continuous support, extensive assistance, and motivating encouragement throughout the entire research journey.

To their panelists, Mr. Mark Kenide Bastasa, Mrs. Amor Dianne Carillo, and Mr. Dayahan Bugao, for their valuable insights, constructive feedback, thorough corrections, professional opinions, and practical suggestions that strengthened and refined this study.

To their parents, guardians, and loved ones, for their steadfast love, financial backing, moral encouragement, understanding patience, and inspiring motivation that sustained them through challenges and helped realize their aspirations.

Finally, to Almighty God, for the divine wisdom, inner strength, and abundant blessings that empowered them to persevere and successfully complete this work.

The researchers are profoundly thankful for each person's invaluable contributions to their achievement.

Researchers

EXECUTIVE SUMMARY

This capstone project focused on the development of **Floodline Alert: An Arduino-Based Flood Warning with SMS Notification** for flood-prone communities in the Municipality of Guipos, Zamboanga del Sur. Recurring flooding caused by river overflows during heavy rainfall remains a major concern in low-lying barangays, where residents often receive limited or delayed warnings. Insufficient flood early warning mechanisms continue to place lives, property, and livelihoods at risk; thus, the system was developed to provide a timely and accessible solution for flood monitoring and early notification.

The system utilizes an ESP32 microcontroller integrated with water level sensors and float switches to continuously monitor rising water levels. Automated SMS alerts are sent to registered users once critical thresholds are reached, ensuring timely warnings even without internet access. Solar panels and power banks are incorporated as backup power sources to maintain continuous operation during power interruptions. The system underwent functionality, stress, reliability, and beta testing to evaluate overall performance and usability.

The Floodline Alert system enhances early warning capabilities and disaster preparedness for both residents and local disaster management authorities. It enables residents to prepare and respond effectively to flood events while providing authorities with a reliable monitoring and communication tool. Overall, the system demonstrates the effective application of affordable technology and renewable energy in reducing flood risks and strengthening community resilience in Guipos, Zamboanga del Sur.

TABLE OF CONTENTS

Content	Page
APPROVAL SHEET	ii
ACKNOWLEDGEMENT	iii
EXECUTIVE SUMMARY	iv
TABLE OF CONTENTS	v
CHAPTER I: BACKGROUND OF THE STUDY	
Introduction	1
Project Context	2
Purpose and Description	3
Objectives	4
Scope and Limitations	5
Definition of Terms	7
CHAPTER II: REVIEW OF RELATED STUDIES AND SYSTEMS	
Related Studies and Systems	12
Foreign	12
Local	16
CHAPTER III: DESIGN AND METHODOLOGY	

Design and Methodology 19

Requirements Analysis 19

Discover Phase 20

Design Phase 22

Develop Phase 29

Deliver Phase 38

CHAPTER IV: DEVELOPMENT AND TESTING

Description and Prototype 40

Development 42

Sensor Readings Module 42

SMS Alert Module 47

Testing 52

Software (Website) Development 52

Website Role in System Control and Performance 52

Data Management, Integration, and Safety 53

ESP32 Development 54

Hardware Initialization and Sensor Validation 54

System Integration and Alert Logic Validation 55

Performance, Stability, and Safety Validation 57

Implementation Plan 59

CHAPTER V IMPLEMENTATION RESULT, DISCUSSION, CONCLUSION AND RECOMMENDATION

Implementation Result and Discussion 60

Conclusion 64

Recommendation 66

REFERENCES 67

APPENDICES

A. Test Cases 69

B. Storyboard 75

C. Sample Input/Output 77

D. Photo Documentation 78

E. Curriculum Vitae 82

LIST OF FIGURES

Figures **Page**

1. 4D Model 20
2. Current Technical Situation 21
3. Context Diagram 22
4. Conceptual Framework Diagram 23
5. Activity Diagram 24
    a. Admin 24
    b. Resident 25
6. Use Case Diagram 26

Entity Relationship Diagram 28

7. Structured Diagram 29
    a. Admin 29
    b. Resident 30
8. Flowchart 30
    a. Float Switch 31
    b. Water Level Sensor 32
    c. ESP32 Microcontroller 33
    d. Admin Dashboard 33
9. Circuit Diagram 37
10. System Architecture Diagram 38

LIST OF TABLES

Tables **Page**

1. Use Case Description 27

2. Component List 34

3. Gantt Chart 40

4. Implementation Plan 59

LIST OF IMAGES

Images **Page**

1. Evacuation Route Map of Guipos 5
2. Flood Susceptibility Map of Guipos 6
3. Data Gathering 20
    a. Interview 20
    b. Audio Recording 21
4. Wiring Diagram 36
5. System UI 60
    a. Admin Homepage 61
        i. Resident Management 61
        ii. ESP32 Sensors 62
        iii. Sensor Readings 62
        iv. SMS Alert 63
    b. Resident Homepage 63
    c. Floodline Alert ESP32 Prototype 64

CHAPTER I

THE PROBLEM AND ITS SCOPE

Introduction

Flooding is a persistent global threat affecting millions of people every year, particularly in the Philippines where frequent typhoons and heavy rains often lead to devastating impacts on communities especially in rural and low-lying areas. According to Engineer Renan F. Mangle the MGDH of the LDRRMO of the Municipality of Guipos in his interview he states that, “In the past, a significant flood occurred in Bagong Oroquieta - “Bugwak”, Guipos, Zamboanga del Sur, causing extensive damage and necessitating rescue efforts by the Local Disaster Risk Reduction and Management Office (LDRRMO)”, this illustrates the vulnerability of these communities to flood disasters. Despite the efforts by municipal agencies like the LDRRMO to implement various disaster preparedness strategies, many rural and low-lying areas remain highly vulnerable. Due to the lack of timely flood warnings these communities suffer repeated loss of property and endangerment of lives.

Recently, many low-cost flood warning systems have been created using small, affordable devices called microcontrollers. These systems use ultrasonic water level sensors and float switches to check water levels and send quick alerts through text messages (SMS) to people’s phones, even if they do not have internet. Some of these systems use solar power to keep working during power outages, and others use different kinds of sensors to make sure the warnings are accurate. They also send different levels of warnings depending on how serious the flood is.

Additionally, some systems integrate advanced data processing techniques, such as machine learning models, to improve the accuracy of flood predictions and provide more reliable alerts. These kinds of systems show how simple technology can

help communities get early flood warnings, especially in places that do not have good internet or expensive equipment. This helps people prepare better and stay safe during floods, making flood monitoring more accessible, sustainable, and effective. (Abalos & Fajardo, 2023; Nur Anis Athirah et al., 2020; Nurzaid Muhd Zain et al., 2020).

Inspired by these modern solutions, a locally adapted flood warning system has been developed titled “Floodline Alert: Arduino-Based Flood Warning with SMS Notification” to continuously monitor water and automate sending SMS alerts to registered users once danger levels are reached. To maintain functionality during power interruptions the system has a solar panel and/or power bank as backup. A digital dashboard further supports community preparedness by displaying evacuation maps, emergency contact numbers, and safety reminders. Using available technologies and renewable energy, the system provides vulnerable communities with the tools they need to enhance flood resilience and safeguard lives and livelihoods.

Project Context

In the Municipality of Guipos, Zamboanga del Sur, frequent flooding caused by the overflowing river during heavy rains presents a serious challenge to residents. The area includes several low-lying barangays where rising water levels quickly threaten homes, farms, and livelihoods. Due to the absence of an effective early warning system, people often have short time to prepare or evacuate, resulting in repeated property damage, loss of income, and risk to lives. Although agencies like the Local Disaster Risk Reduction and Management Office (LDRRMO) and local barangay officials have implemented disaster preparedness measures, these efforts have not fully addressed the vulnerability of these communities. This situation highlights the urgent need for a simple and reliable flood warning system tailored to the local context.

This project aims to develop the *"Floodline Alert: Arduino-Based Flood Warning System with SMS Notification"* to fill this gap. The admin can monitor water levels and sends immediate flood alerts via SMS to residents, ensuring timely warnings even for those without internet access or smartphones. By providing early notifications, the community can better prepare, evacuate if necessary, and protect their properties and livelihoods. The inclusion of solar panels and power banks as backup power sources guarantees continuous operation during power outages, which are common during severe weather.

This project is timely and important as it leverages affordable technology and renewable energy to enhance disaster resilience, contributing positively to public safety, environmental sustainability, and technological innovation. The primary beneficiaries of this system are the residents of flood-prone barangays in Guipos, who will gain access to reliable flood warnings and emergency information. Local government units and disaster response teams will also benefit from improved community preparedness and communication. In summary, the Floodline Alert system is a practical, community-focused solution designed to provide continuous flood monitoring, instant SMS alerts, and a digital dashboard featuring evacuation maps and emergency contacts, empowering vulnerable communities to respond effectively to flood threats.

Purpose and Description

The primary purpose of the Floodline Alert system is to provide timely and reliable flood warnings to residents in flood-prone areas of Guipos, Zamboanga del Sur. By monitoring water levels and sending automated SMS notifications, the system aims to give communities enough time to prepare, evacuate, and protect their homes and livelihoods. This project addresses the critical need for an accessible early warning

solution that works even in areas with poor internet connectivity and frequent power outages, ultimately helping to reduce loss of life and property during flood events.

The Floodline Alert is an innovative flood warning system using Arduino (ESP32) microcontrollers connected to water level sensors and float switches to monitor flood-prone areas continuously. When water reaches a critical level, the admin sends automated SMS alerts to registered users, ensuring warnings reach residents without relying on internet or smartphones. Its unique use of solar panels and power banks ensures continuous operation during power outages common in severe weather. The system also includes a digital dashboard with evacuation maps, emergency contacts, and safety tips, boosting community preparedness. This reliable and user-friendly design makes Floodline Alert an effective solution for vulnerable rural communities.

Objectives

General Objective

To develop a reliable and cost-effective flood warning system that provides timely SMS alerts and enhances disaster preparedness for flood-prone communities in Guipos, Zamboanga del Sur.

Specific Objectives

This project specifically aims:

1. To design an Arduino-based flood monitoring system integrated with water level sensor or float switch and GSM modules to detect rising floodwaters accurately.
2. To develop an automated SMS notification feature that sends real-time flood alerts to registered users, ensuring accessibility even without internet connectivity.

3. To implement a solar-powered backup system using solar panels and power banks to maintain uninterrupted operation during power outages.
4. To test the complete Floodline Alert system in selected barangays of Guipos to evaluate its accuracy, reliability, and user responsiveness.

Scope and Limitations

The designated environment for this project is located within the municipality of Guipos, Zamboanga del Sur, which serves as the broader geographical context of the study. Specifically, the selected barangay within this municipality has been identified as highly susceptible to flooding, thereby necessitating focused analysis and intervention in relation to disaster risk management and community resilience.



Image 1. Evacuation Route Map of Guipos

The image 1 shows the Evacuation Route Map of Guipos, a municipality in the province of Zamboanga del Sur, Region IX, Philippines. The map illustrates the municipal boundary along with road networks and the locations of designated evacuation centers, which are marked by red starburst icons. These evacuation centers are distributed across various barangays to ensure accessibility for residents during emergencies, particularly natural hazards such as flooding.

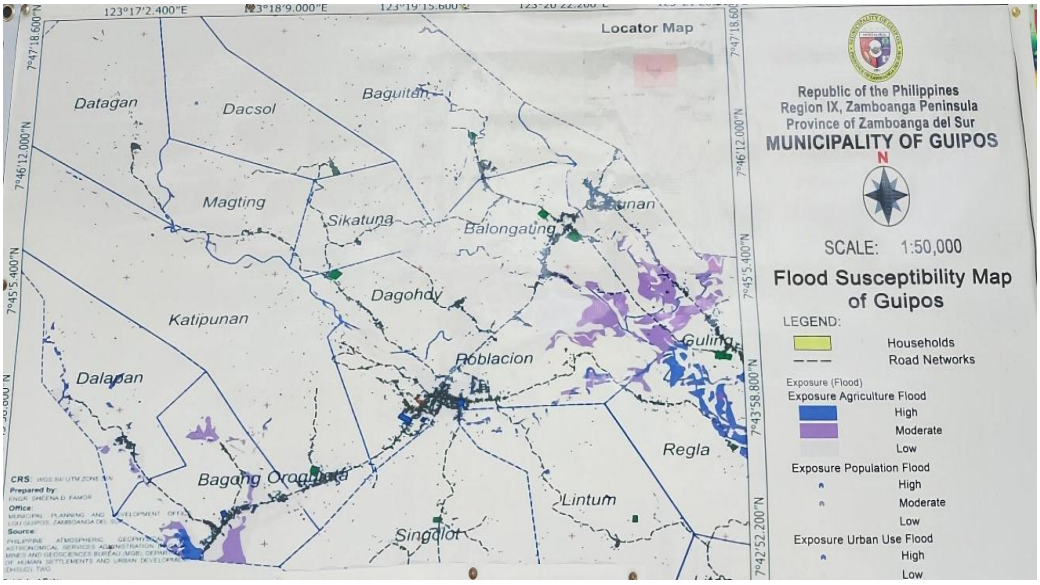


Image 1.1. Flood Susceptibility Map of Guipos

The image 1.1 presents the Flood Susceptibility Map of Guipos, a municipality in Zamboanga del Sur, Philippines. It identifies areas that are prone to flooding, with color codes indicating the degree of susceptibility: high (dark purple), moderate (lighter purple), and low (uncolored areas). The map also marks households, road networks, and agricultural lands, showing the level of exposure to flood risks.

This project focuses on designing and developing the Floodline Alert system to provide real-time flood warnings through SMS notifications for selected flood-prone barangays according to Engineer Renan F. Mangle in his interview “the most affected areas during flooding are: Bugwak, Bagong Oroquieta, Poblacion, Dagohoy, Magting, Sikatuna, Dacsol, Datagan, Balongating, Canunan, Regla, and Guling in the Municipality of Guipos, Zamboanga del Sur “.

The system continuously monitors water levels using Arduino-based sensors and float switches and sends SMS alerts to registered residents. To maintain functionality during power outages, the project includes the integration of solar panels and/or power banks as backup power sources. Additionally, a digital dashboard will be developed to display evacuation maps, emergency contacts, and safety tips.

However, this project does have certain limitations. Through testing and observation, the researchers determined that the ESP32 microcontroller sends data logs to the database every 10 seconds in the span of 6 minutes. This interval was chosen because further reducing the time between transmissions caused the system to crash. Therefore, a 10 second interval is sufficient for accurate water level data analysis while ensuring the system remains stable and avoids overload.

In addition, this project does not include flood prediction or weather forecasting beyond the monitoring of water levels, nor does it integrate with national or regional disaster management systems.

The SMS alert feature relies on the availability of cellular networks, which may be unreliable in some remote areas. Furthermore, the system is designed primarily for small to medium-scale flooding events and may not be effective for sudden or large-scale floods. Maintenance and repair of hardware after deployment are beyond the scope of this project and will depend on local support.

Lastly, while the digital dashboard provides valuable information to the community, it is not intended to replace official government disaster communication channels.

Definition of Terms

Floodline Alert: Arduino-Based Flood Warning with SMS Notification. This term refers to the Arduino-based flood warning system developed in this study that monitors water levels and sends SMS notifications to residents in flood-prone areas.

ESP32. This term refers to an open-source microcontroller platform used in the project to control sensors and automate flood detection and SMS alerts.

SMS (Short Message Service). This term refers to the text messaging function used by the system to notify residents about water level changes or potential flooding.

Float Switch / Water Level Sensor. This term refers to the device used to monitor the water level in real-time and trigger alerts when it reaches a critical height.

Solar Panel. This term refers to the renewable energy component used to power the system during daylight and charge backup sources for use during power outages.

LoRa (Long Range). This is a wireless technology that allows long-distance, low-power data transmission, ideal for sending flood sensor data in remote areas without relying on cellular networks.

Internet of Things (IoT). This refers to a network of connected devices with sensors that collect and share data, enabling real-time flood monitoring and efficient alerting in this project.

CHAPTER II

REVIEW OF RELATED STUDIES AND SYSTEMS

The Floodline Alert System is grounded in a solid technical framework that brings together carefully selected hardware and software components within an integrated programming environment. Supported by relevant studies from both local and foreign sources, the system's design emphasizes adaptability and functionality to effectively address flood monitoring needs.

Technical Background

The Floodline Alert is a project that helps communities stay safe by giving early warnings about rising floodwater. It uses a small smart controller, called an ESP32, with a water sensor and float switch to constantly check water levels. When water rises too high, the system sends automatic text message alerts to warn people in time to prepare. Powered by solar panels and batteries, it works even without electricity, and its parts are protected in a waterproof box to handle rain and harsh weather. Designed with common, easy-to-replace components, it is reliable and easy to maintain, making it a practical tool for flood-prone areas to stay safe and ready.

System Overview

Floodline Alert utilizes a combination of hardware and software components to monitor water levels and automatically send SMS alerts to registered users when dangerous thresholds are detected. The system is built around the ESP32 microcontroller platform, which processes data from connected water level sensors or float switches. When a flood risk is detected, the ESP32 communicates with a HTTPSMS gateway to send automated SMS notifications to residents, ensuring rapid

dissemination of warnings even in areas where internet access is unreliable or unavailable. To guarantee continuous operation during power outages, the system is equipped with a solar panel and/or power bank as backup power sources. A digital dashboard, accessible via local network or display, provides evacuation maps, emergency contacts, and safety reminders to further support community preparedness.

Hardware Specifications

Hardware Component	**Description**
ESP32 Dev Board	Main microcontroller (handles sensors, logic, communications; 3.3V logic)
Solar Panel	Provides renewable energy for backup power
Power Bank	Portable battery backup to ensure uninterrupted operation
ESP32 Dev Board	Main microcontroller (handles sensors, logic, communications; 3.3V logic)
Water Level Sensor – Analog pin	Water level sensor (3.3V, requires voltage, and output (Analog pin – S) to ESP32 via GPIO32 pin)
INA219 Current/Voltage Sensor	Optional, for monitoring system power consumption and battery voltage
PowMr 20W Solar Panel	Main renewable power supply
12V 7Ah/9Ah VRLA or 4x 32650 LiFePO4	Rechargeable battery for continuous operation
PowMr 30A PWM Charge Controller	Regulates solar charging and protects battery
LM2596 DC-DC Buck Module	Converts battery voltage to stable 5V for ESP32 and GSM module
Resistor, 20kΩ (voltage divider)	For 3.3V-safe logic level at ESP32 inputs
Common Ground Bus or Rail	Ensures all modules share a common, reliable ground

IP65 Waterproof Box/Enclosure	Protect electronics outdoors
Breadboard or Custom PCB	Assembly platform: PCB recommended for field reliability
Jumper Wires, Connectors	For all signal and power wiring

Software Specifications

Software Component	Description
Arduino IDE	Programming environment for developing and uploading code to the Arduino/ESP32 with embedded C++ as the programming language used for system logic and sensor integration
SMS Gateway Library	Software library for interfacing with the HTTPSMS
HTTPSMS	An SMS API Gateway Software which will send the Automated SMS to residents
Digital Dashboard	Local web interface or display for evacuation maps and emergency info

Programming Environment

The development of the Floodline alert system was accomplished through the integration of both robust hardware and versatile software elements, carefully tailored to facilitate real-time flood monitoring, automated alerts, and user interaction.

The system uses the ESP32 Dev Board as the central microcontroller, selected for its advanced processing capabilities, integrated WiFi/Bluetooth, and support for a wide range of sensors and communication modules.

Essential hardware elements include an analog water level sensor and float switch for consistent, real-time measurement of water levels, and renewable energy solutions such as a solar panel and backup power bank to ensure continuous operation even during power outages.

On the software front, development was conducted using the Arduino IDE, the standard programming environment for ESP32 and similar microcontroller platforms. All logic and sensor interaction were implemented in Embedded C++, which allows for efficient and precise control of hardware, sensor data acquisition, and communication routines. Key libraries required for project functionality included an SMS gateway library, enabling streamlined sending of emergency SMS alerts via the HTTPSMS Gateway GSM, and additional utilities for sensor integration and serial communication.

To support user interaction and emergency decision-making, a local digital dashboard was designed as a web-based interface, capable of displaying live flood data, system status, and evacuation maps. This dashboard ensures that critical information is accessible in real time by first responders or local authorities. Together, these hardware and software tools provided a reliable, always-on solution for flood monitoring and public safety alerting in vulnerable areas.

Related Studies and Systems

Foreign

Recent research and system development abroad have increasingly recognized the importance of SMS-based flood warning systems, especially in places that are flood-prone areas. These systems use mobile networks to deliver timely alerts, making them practical for remote communities.

A Real-Time Flood Monitoring and Alert System: Enhancing Community Resilience through Technology Integration

One notable study is by Kamau, G. (2023) a case study of River Tana in Kenya which evaluated the reliability of SMS-based flood alert systems by simulating flood events and measuring the speed of alert delivery. The system architecture included IoT

sensors, predictive analytics, and mobile-based alerts and the system aims to provide a more timely and reliable solution compared to traditional weather forecasts or static flood models.

This will help local authorities respond faster and more effectively to flood events, potentially saving lives and reducing infrastructure damage. Field tests were conducted in both urban and rural settings to assess the responsiveness and coverage of the SMS notifications. The study confirmed that SMS notifications could be sent within five seconds of detecting dangerous water levels, proving the effectiveness of SMS as a rapid and reliable communication tool for early warning in areas with poor internet infrastructure.

This study relates to the project because it emphasizes disaster preparedness using technology in providing real-time flood warnings. It highlights the effectiveness of SMS alerts in ensuring community safety and timely information dissemination during flood events.

Flood Warning and Monitoring System (FWMS) using GSM Technology

In a similar vein, Nurzaid Muhd Zain et al., (2020) developed a personal Flood Warning and Monitoring System (FWMS) utilizing GSM technology to provide real-time flood alerts to residents in flood-prone areas of Malaysia. The system is built around an Arduino Uno microcontroller interfaced with an ultrasonic sensor (HC-SR04) to measure flood water levels and a GSM SIM900A module to send SMS notifications and make alarm calls. FWMS categorizes flood risk into three levels: (“normal”, “warning,” “prepare,” “evacuate,” with SMS alerts and buzzer alarms triggered at the latter two stages to warn residents and emergency services such as the Fire and Rescue Station. Users can also request real-time flood status updates via SMS, enabling

proactive monitoring. The system was tested for functionality, prototype performance, and GSM network reliability, yielding positive results that demonstrate its potential for effective flood monitoring and early warning.

The study highlights the importance of accessible, low-cost flood warning systems that empower residents to respond promptly to rising floodwaters, reducing property damage and loss of life. The FWMS was specifically designed for flood-prone residential areas near rivers, paddy fields, and drainage systems in Malaysia's northern and east coast regions, with pilot implementations in Perlis. This system's integration of ultrasonic sensing with GSM-based communication offers a practical model for early warning, especially in communities with limited internet access.

This study relates to the project because it demonstrates the use of Arduino-based technology integrated with GSM for real-time flood monitoring and early warning dissemination. It highlights the practicality of ultrasonic sensors in measuring water levels and the effectiveness of SMS alerts in delivering categorized flood warnings, which are crucial in enabling communities with limited internet access to prepare and respond promptly during flood events.

Solar-powered flood early warning system with short message service (SMS) notifications

Another further example, Nur Anis Athirah and colleagues (2020) developed a solar-powered flood early warning system designed to provide timely SMS notifications to residents in flood-prone areas of Malaysia. Recognizing the frequent occurrence of floods and the challenges posed by unreliable power sources during such events, the system is powered primarily by solar energy, with a battery and solar charger controller ensuring continuous operation even during nighttime or cloudy conditions.

The hardware setup includes an Arduino Uno microcontroller interfaced with a water level sensor to monitor floodwater heights in real time. The system uses three LEDs to visually indicate different water levels—green for safe, yellow for alert, and red for danger—each corresponding to specific flood thresholds. When water levels reach the alert or danger stages, the system automatically sends SMS notifications via a GSM module to residents, enabling them to take timely precautionary measures.

The researchers tested the prototype extensively, demonstrating that the system accurately detects water levels and reliably sends alerts, even in remote areas with limited infrastructure. This project highlights the importance of integrating renewable energy with IoT and mobile communication technologies to create sustainable, low-cost, and effective flood warning solutions.

The use of SMS ensures that critical flood warnings reach communities regardless of internet availability, addressing a key limitation in many flood-prone regions. This system is particularly relevant for areas like Batu Pahat, Malaysia, which experienced severe flash floods in 2017, underscoring the need for early notification to reduce loss of life and property damage.

This study relates to the project because it demonstrates the effectiveness of a solar-powered flood warning system using Arduino and SMS technology. It emphasizes the importance of renewable energy for uninterrupted operation during power outages and highlights the role of SMS alerts in reaching communities with limited internet access, aligning with Floodline Alert's goal of providing reliable and sustainable flood warnings.

Collectively, these foreign studies demonstrate that SMS-based alert systems are a proven, effective solution for flood-prone regions with poor internet access during

heavy rains. Their success supports the integration of SMS notification features in flood warning projects like Floodline Alert, ensuring broad and equitable access to life-saving information.

Local

In the Philippines, where many rural and low-lying communities face frequent flooding and low internet service during heavy rains, SMS-based flood warning systems have been recognized as highly practical and impactful. Local research and government agencies have focused on using mobile networks to deliver timely flood alerts that can reach even the most remote populations.

Community-based flood alert system using long-range technology for Brgy. San Agustin, San Jose, Occidental Mindoro

A significant example is the community-based flood alert system developed for Barangay San Agustin, San Jose, Occidental Mindoro by Abella and Enriquez (2021). Their system uses Arduino Uno microcontrollers combined with LoRa long-range wireless technology and GSM modules to continuously monitor water levels and environmental conditions. When sensors detect rising floodwater, the system transmits data via LoRa to a central receiver, which then triggers SMS alerts to residents within ten seconds.

This rapid response capability ensures that warnings reach people regardless of internet availability, making it especially suitable for rural communities vulnerable to sudden floods. This study relates to the project because it utilizes Arduino Uno microcontrollers with LoRa wireless technology and GSM modules to provide rapid flood monitoring and SMS alerts. It highlights the system's effectiveness in delivering timely warnings within seconds, particularly in remote communities without internet

access, aligning with Floodline Alert's objective of enhancing preparedness and safety in flood-prone areas through accessible technologies.

Real Time Flood Detection, Alarm and Monitoring System Using Image Processing and Multiple Linear Regression

Another local study by Tolentino, Reyes, and Santos (2021) developed a real-time flood detection, alarm, and monitoring system for flood-prone areas in the Philippines. Their system uses a rain gauge, float switch, and flow rate meter sensor connected to an Arduino microcontroller to continuously monitor rainfall, water level, and river flow. Data are processed in real time, triggering alarms and flood warnings when thresholds are met. Field tests in Bulacan showed high reliability and accuracy, comparable to manual methods. By automating data collection and alerts, the system reduces response time and improves community preparedness, offering a practical solution for local disaster management.

This study highlights the effectiveness of affordable sensors and microcontrollers in enhancing early warning systems in vulnerable areas. The study is relevant to the Floodline Alert project as it showcases a real-time flood detection and alarm system using Arduino microcontrollers and multiple sensors to monitor rainfall, water levels, and river flow.

Their approach automates flood monitoring and alerts with high accuracy and reliability, reducing response times and improving community preparedness in flood-prone areas of the Philippines.

This reflects the Floodline Alert's aim of using affordable sensor technology and microcontrollers to provide timely flood warnings and enhance disaster risk management in vulnerable communities.

IoT based flood detection, alarm and monitoring system using multilayer perceptron and regression

Lastly, Abalos and Fajardo (2023) from Isabela State University developed an IoT-based flood detection, alarm, and monitoring system that significantly improves the accuracy and timeliness of flood predictions by integrating sensor data. Their system uses an Arduino UNO microcontroller connected to multiple sensors, including Light Detection and Ranging (LiDAR) for precise flood level measurement in feet, a rain gauge for measuring precipitation rate in millimeters per hour, and a flow rate meter to monitor river flow fluctuations in liters per hour. The collected data is processed in real time and sent via SMS notifications to residents in flood-prone areas, enabling timely evacuation and risk mitigation.

This study relates to the project because it introduces an IoT-based flood monitoring system using Arduino UNO with multiple sensors, including LiDAR, a rain gauge, and a flow rate meter. It highlights the use of predictive models and SMS alerts to provide accurate and timely flood warnings, aligning with Floodline Alert's goal of enhancing community preparedness through technology-driven solutions.

These local studies and initiatives confirm that SMS-based flood warning systems is effective in the Philippine context. They ensure that critical alerts reach even the most remote and underserved populations, directly supporting the goals and design of the Floodline Alert project.

CHAPTER III

DESIGN AND METHODOLOGY

The design and methodology of the Floodline Alert System follow a structured and systematic approach to ensure reliability and effectiveness. By employing a comprehensive model that emphasizes refined analysis, detailed design, precise development, and thorough delivery, the system integrates both hardware and software components to provide accurate and timely flood alerts. This methodology supports flexibility and continuous refinement throughout the project lifecycle.

Requirement Analysis

According to Float (2025) and Simplilearn (2025), the 4D Model (Discover, Design, Develop, Deliver) is a flexible framework that supports early testing and continuous user feedback. The researchers adopted this model for the Floodline Alert System because it allows effective integration of hardware and software components. In contrast, the Waterfall method is more rigid and limits revisions, making it less suitable for complex Arduino (ESP32)-based projects that require adaptability (EPAM Startups & SMBs, 2024; GeeksforGeeks, 2025; Forbes Advisor, 2024).



Figure 1. 4D Model

Figure 1. shows the 4D model, an effective approach used to guide the development and successful completion of the system. By providing a structured, step-by-step method, this model ensures that each stage from gathering requirements to

deploying the final product is carefully planned and efficiently carried out, making it essential for building a reliable and efficient system.

Discover Phase

In the discovery phase, the researchers identified the key challenges faced by the community. This process included gathering information and conducting interviews for thorough analysis. Based on the data collected from the Municipality of Guipos, particularly through the Local Disaster Risk Reduction and Management Council Office (LDRRMO), the researchers were able to clearly define the problem and propose an appropriate solution: the development of an Arduino-based flood warning system with SMS notifications.

Data gathering



Image 2.1 Interview

Image 2.1 shows one of the researchers conducted an interview with Engr. Renan F. Mangle the head of office of the Local Disaster Risk Reduction and Management Council Office (LDRRMO). The results highlighted a clear need for a rapid and dependable early-warning system.

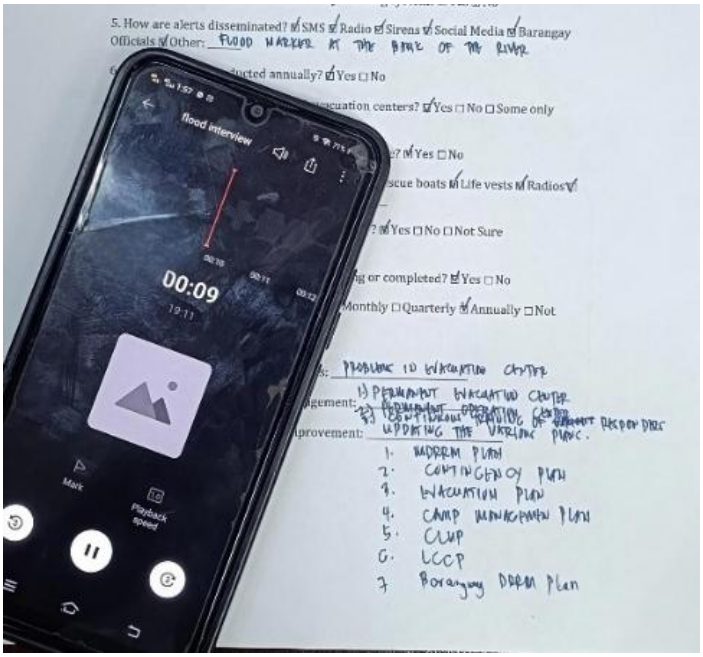


Image 2.2 Audio Recording and Questionnaires

The image 2.3 shows the audio recording from Engr. Renan F. Mangle, and questionnaires to different individuals who participated, including LDRRMO officers and residents of the Municipality of Guipos, to gather insights into the flood risks faced by the community. After analyzing the data and interviews. The researchers developed an Arduino-based flood alert solution featuring automated SMS notifications (See Appendix D).

Current Technical Situation

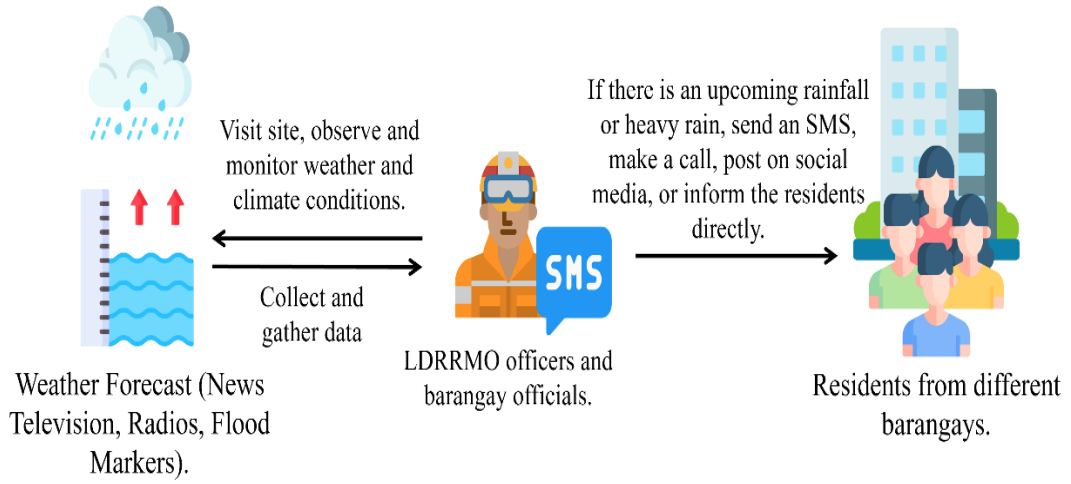


Figure 3. Current Technical Situation

The figure 3, shows how LDRRMO officers and barangay officials currently share flood information via SMS, calls, social media, or direct contact, gathering data from flood markers, radio, and TV. To enhance timely and reliable dissemination, the researchers proposed an automated SMS alert system to deliver flood warnings during flood events.

Design Phase

The system was designed to be functional and user-friendly, enabling LDRRMO admins to send automated SMS updates to residents about their barangay's flood status.

Context Diagram

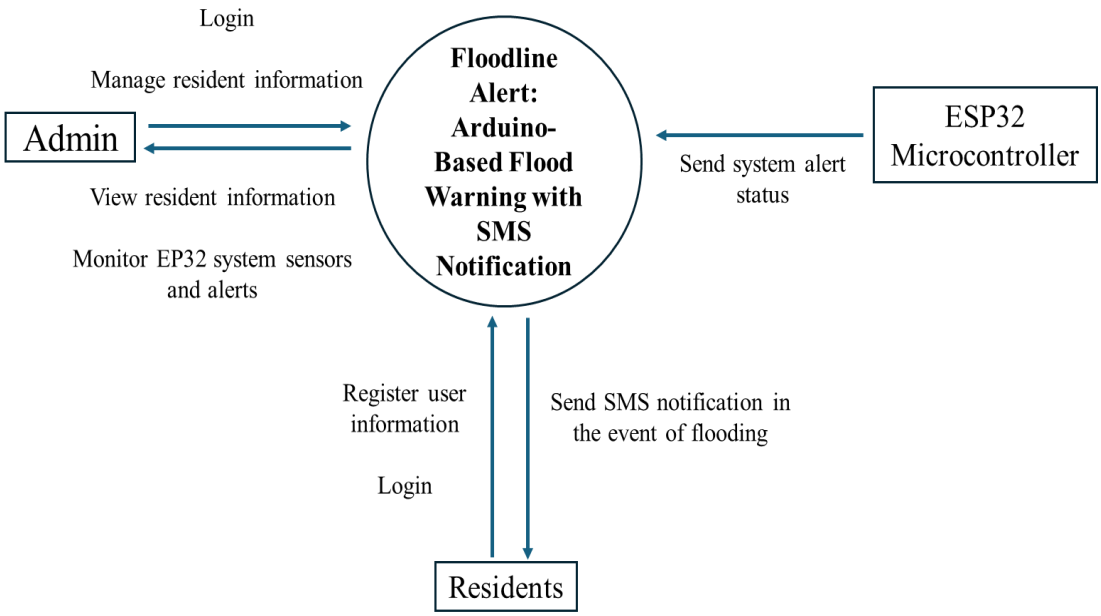


Figure 4. Context Diagram

The figure 4, visually depicts the Floodline Alert System's structure and workflow. The system links the admin dashboard, resident portal, monitoring hardware (ESP32 microcontroller with sensors.

The admin manages resident information's and monitors real-time data such as water level, float switch status.

Residents register, update their information, view profiles, and access news or support through the portal. When sensors detect flooding, the admin triggers the HTTPSMS gateway to send SMS alerts to all registered residents, ensuring timely information delivery even during local network disruptions.

Conceptual Framework

This study aims to develop an Arduino-based microcontroller flood warning system with SMS notification to serve as an early-warning mechanism for the Municipality of Guipos. The system is designed to automate flood alerts, enabling timely dissemination of information to residents and supporting the efforts of LDRRMO officers and barangay officials in disaster preparedness.

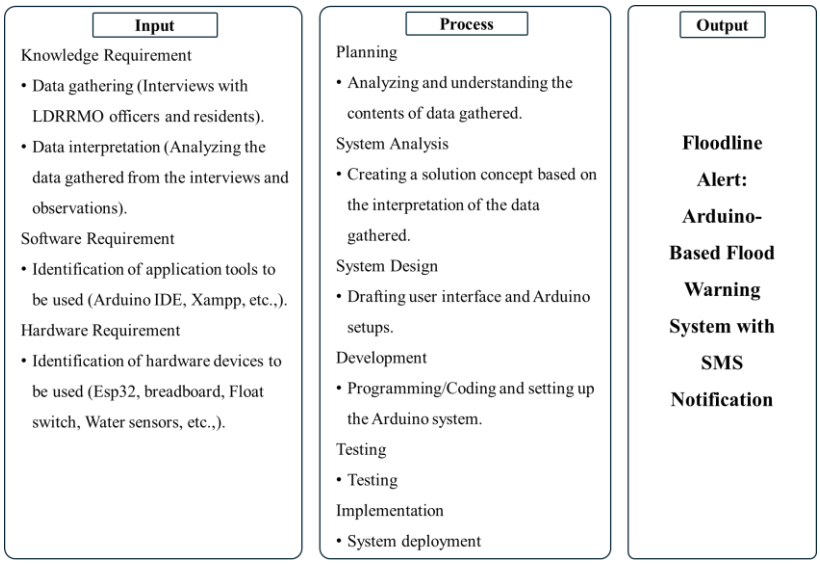


Figure 5. Conceptual Framework Diagram

The Floodline Alert System development started with interviews of LDRRMO officers and residents to identify community needs and challenges. Using this data, the researchers chose software tools (Arduino IDE, XAMPP) and hardware components (ESP32, float switch, water sensors), then designed, programmed, and set up the Arduino-based system. After testing and deployment, the final product was Floodline Alert: an Arduino-Based Flood Warning System with SMS Notification.

Activity Diagram

The activity diagram outlines the workflow of Admin and Resident interactions during normal use and flood events, showing user actions, system decisions, and automated responses clearly and simply.

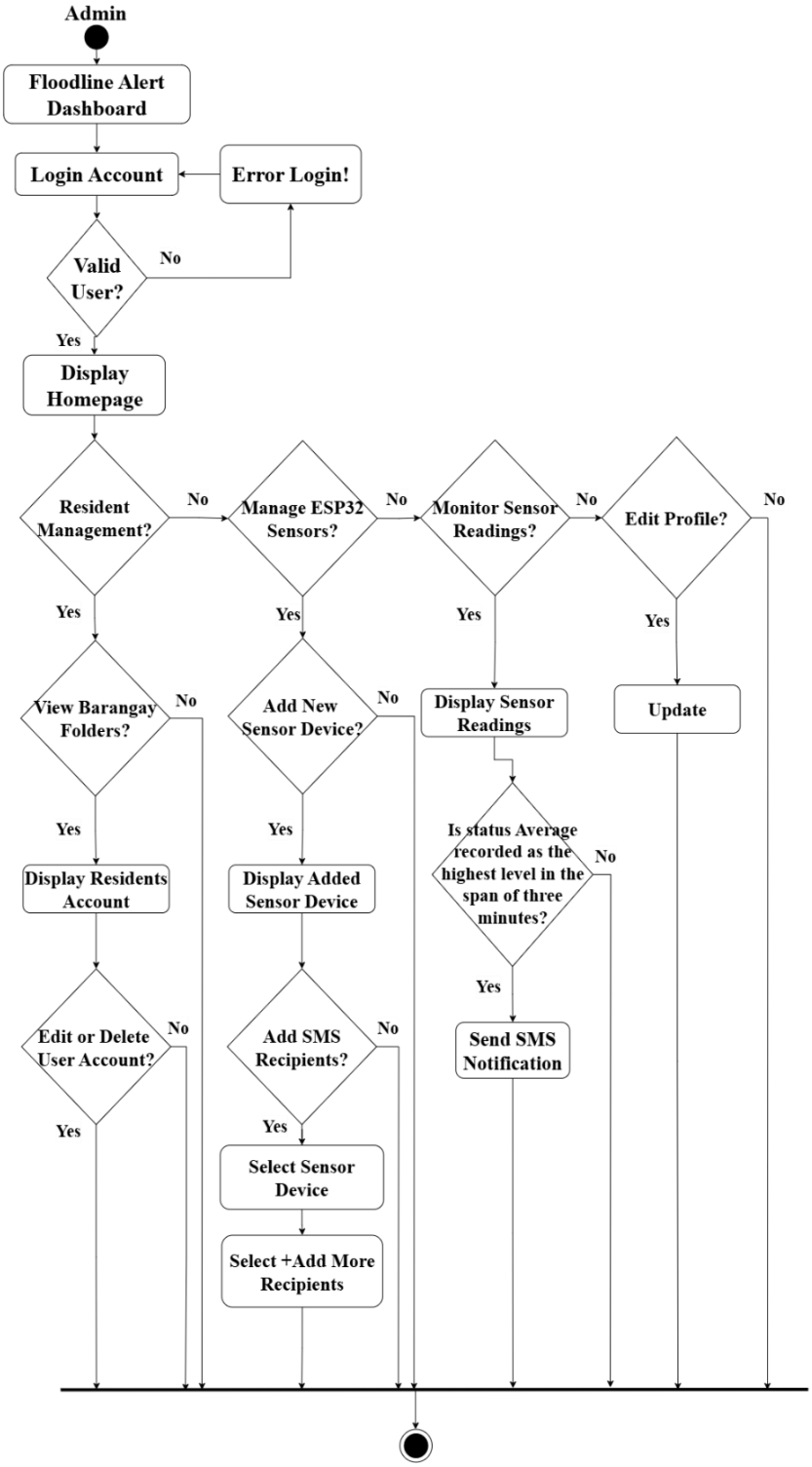


Figure 6.1 Activity Diagram (Admin)

Figure 6.1 shows the admin's workflow, beginning with login and error handling. After successful login, the admin can manage resident info, barangay details, accounts, and delete records. The admin can monitor sensor data and system status, including water levels averaged over 3-6 minutes for accuracy. Based on alert levels ("normal", "warning," "prepare," "evacuate"), the admin can send SMS notifications to residents. The admin can also update their profile.

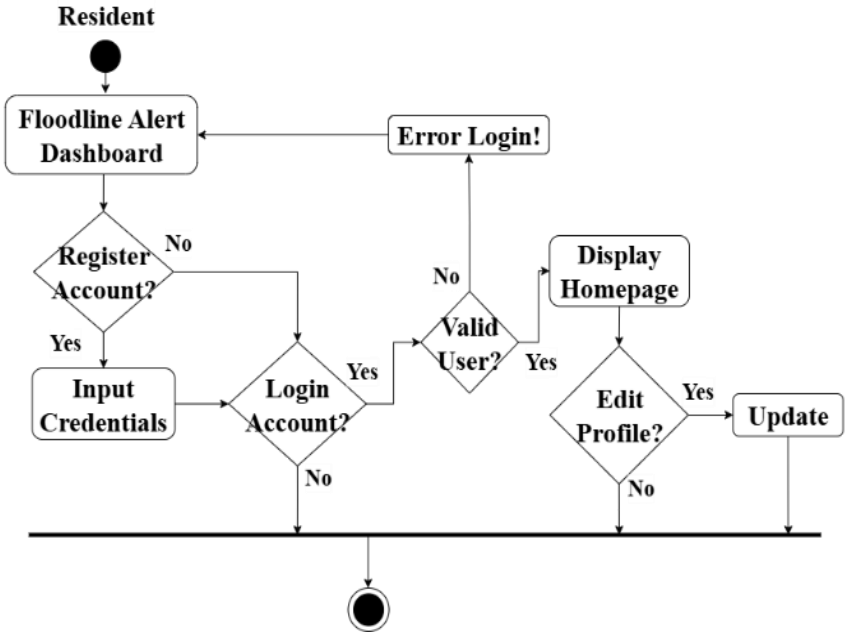


Figure 6.2 Activity Diagram (Resident)

This activity diagram shows the resident workflow, starting with account creation and login. On successful login, residents see the homepage and can update their profiles; if login fails, an error message appears. It highlights account management, system monitoring, and profile updates in the Floodline Alert System.

Use Case Diagram

The use case diagram provides a high-level visual summary of the main ways users interact with the Floodline Alert System. It highlights the system's key functions and clarifies which roles: Admin, Resident, and Sensors can access or initiate each feature.

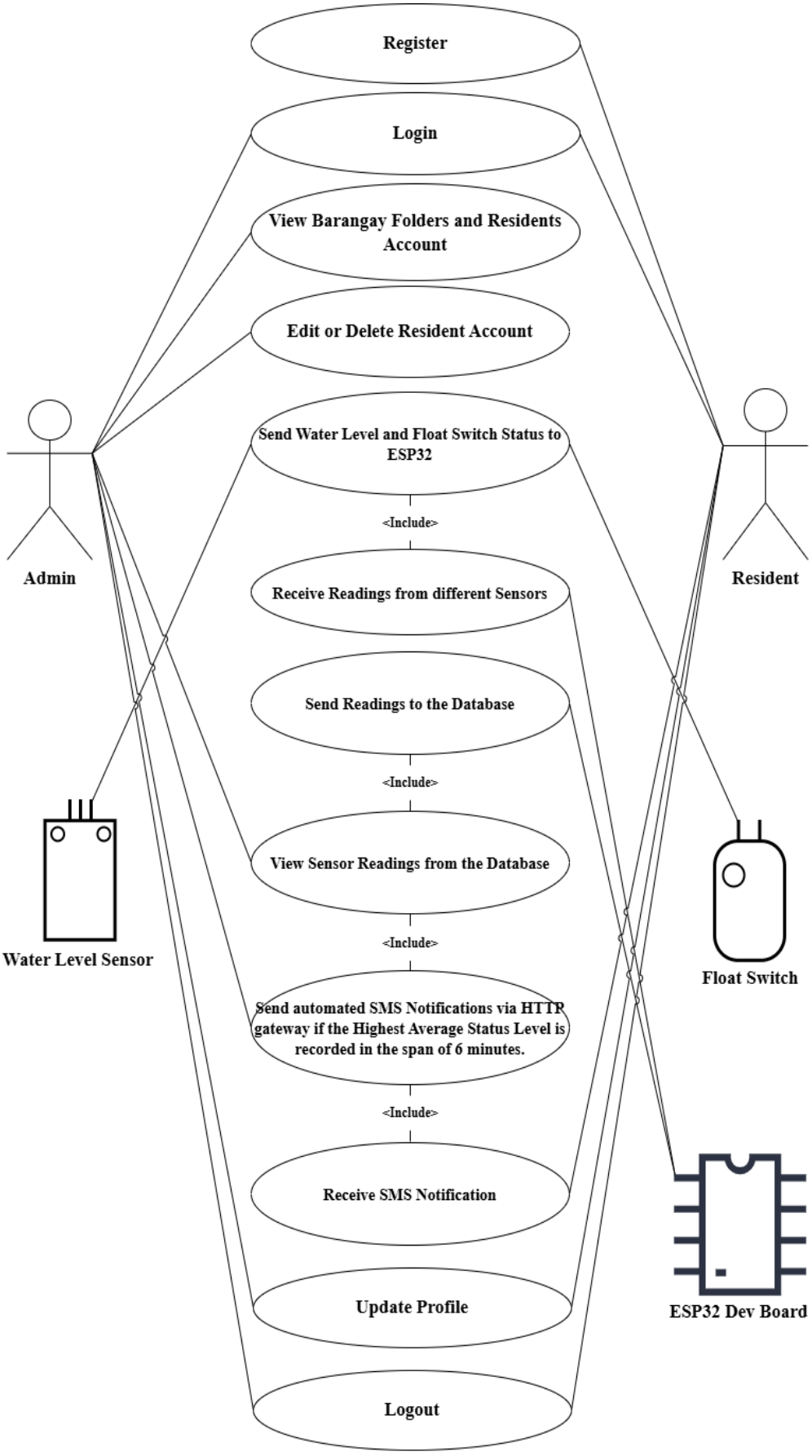


Figure 7. Use Case Diagram

The figure 7, shows how the water level monitoring and notification system supports community safety. The admin logs in to manage resident accounts and view sensor data sent in real time by water level sensors and float switches to the ESP32 microcontroller. When water levels exceed a critical threshold over 6 minutes, the admin can trigger SMS alerts.

The admin sends automated SMS notifications via the HTTPSMS gateway to residents, who receive timely flood warnings. Residents can also register, log in, update profiles, and log out. This system ensures effective coordination between users and technology for prompt, automated alerts.

Use Case Description

A use-case description provides a detailed, step-by-step narrative of how a user achieves a specific goal within the system. It defines the actors involved, the pre- and post-conditions, and both the standard and alternative flows of events. This structured format ensures all scenarios, including exceptional cases, are understood and addressed.

Attribute	Description
Actor(s)	Resident, Admin
Pre-condition	The system is operational; residents are registered
Main Flow	1. System monitors water level sensors. 2. Water exceeds the flood threshold. 3. System composes and sends SMS alert to all residents. 4. Residents receive the flood alert on their registered cell phones. 5. Admin is also notified and can log in for further action.
Alternative Flow	If the SMS fails to send, the system retries after a set interval.
Post-condition	Residents are informed of impending flood risk; Admin can track alert level status and sensor readings.

Table 8. Use Case Description

This use-case details the automatic and manual steps involved in alerting residents during a flood event.

The process starts with real-time sensor monitoring. Once a danger threshold is detected, the system sends SMS notifications to all registered residents and informs the Admin. If any step fails such as SMS delivery the system has contingency flows, like retrying and logging failures, to ensure maximum reliability and traceability of alert dissemination.

Entity-Relationship Diagram

The Entity Relationship Diagram (ERD) shows the data structure and information flow within the Floodline Alert System, combining hardware (ESP32 and sensors), embedded software, and a web dashboard.

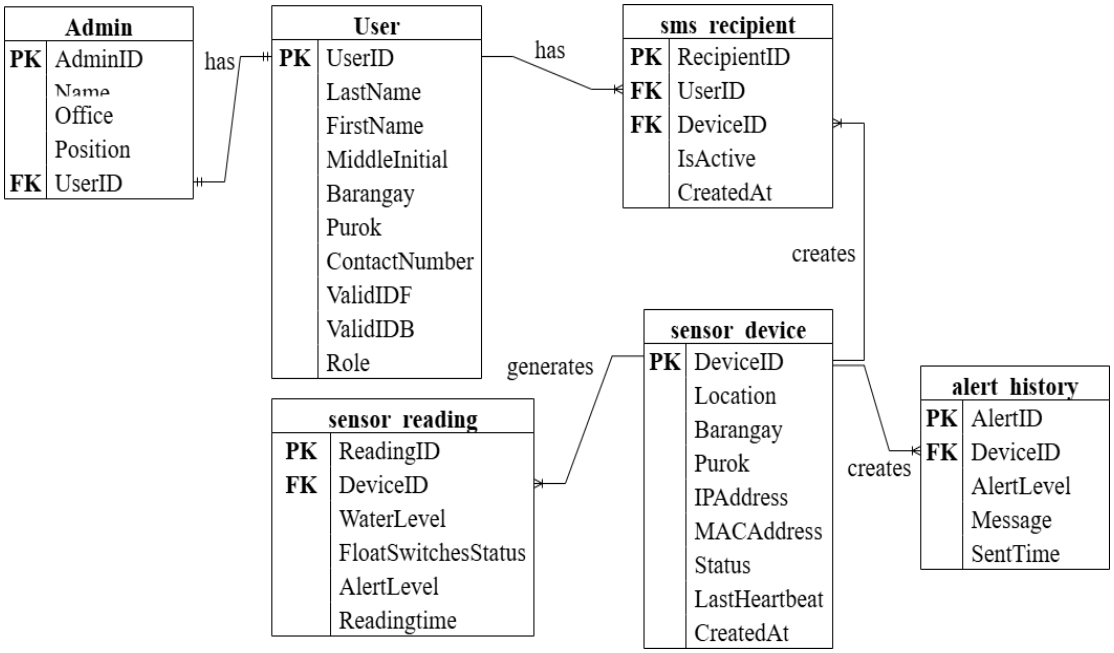


Figure 9. Entity-Relationship Diagram

The figure 9 shows system main entities include User, which contains accounts for residents and admins, linked to Admin record. SMS_Recipient logs device status and is associated with multiple Sensor_Device entries and Sensor_Reading generated during flood events. Alert_History tracks all message exchanges, such as SMS alerts.

These relationships enable efficient management of user data, device monitoring, and notifications throughout the system.

Develop Phase

During development, the researchers integrated hardware and software features of the Floodline Alert System.

The ESP32-based system collects real-time sensor data, manages user registrations, and automates SMS flood alerts.

Structured Diagram

The structured diagram shows the system's hierarchy and module interactions, illustrating how the user interface, microcontroller, communication, and power supply work together to deliver early flood warnings. It clarifies data flow and processes, making the design easier to understand.

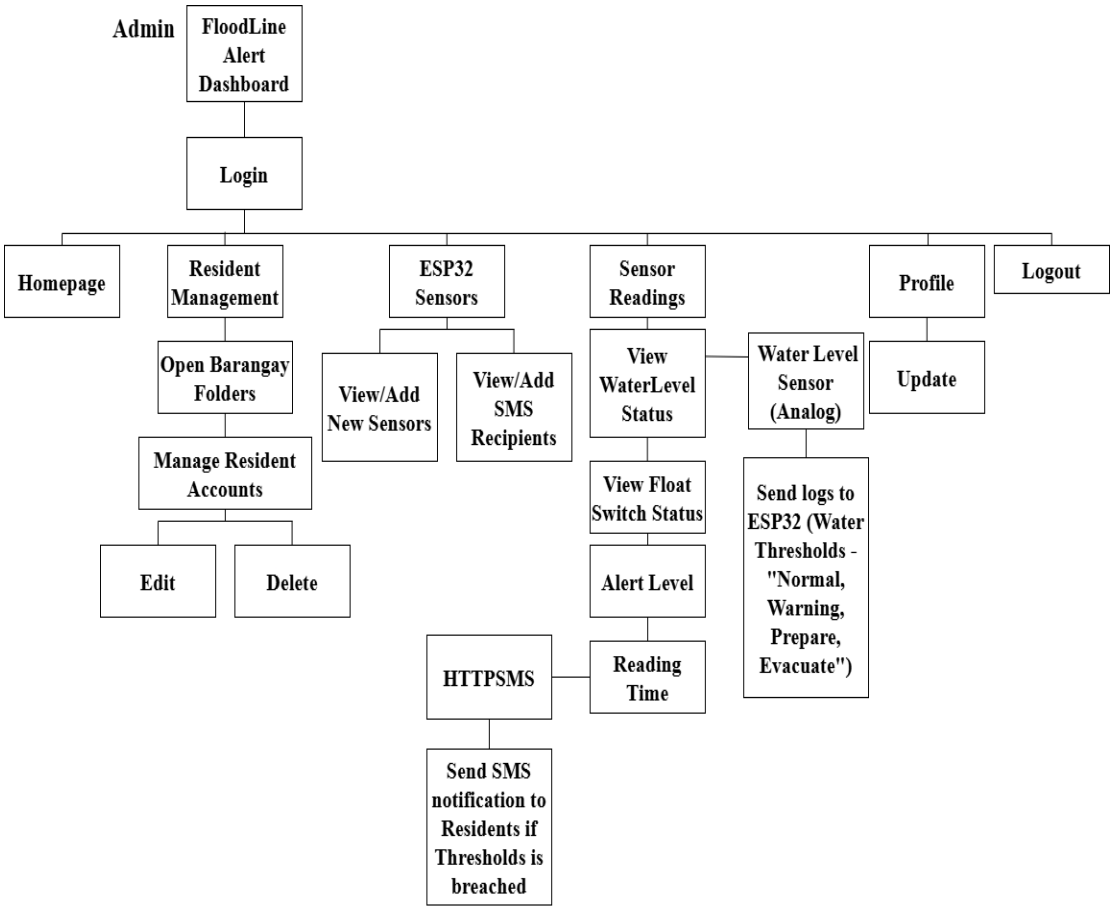


Figure 10.1 Structured Diagram (Admin)

The figure 10.1 shows the system's hierarchy and module relationships. The admin's User Dashboard manages registration, profiles, and alert monitoring.

It connects to the ESP32 Microcontroller, which processes data from the analog Water Level Sensor. When a threshold is met, the admin can send SMS notifications to residents via the HTTPSMS gateway.

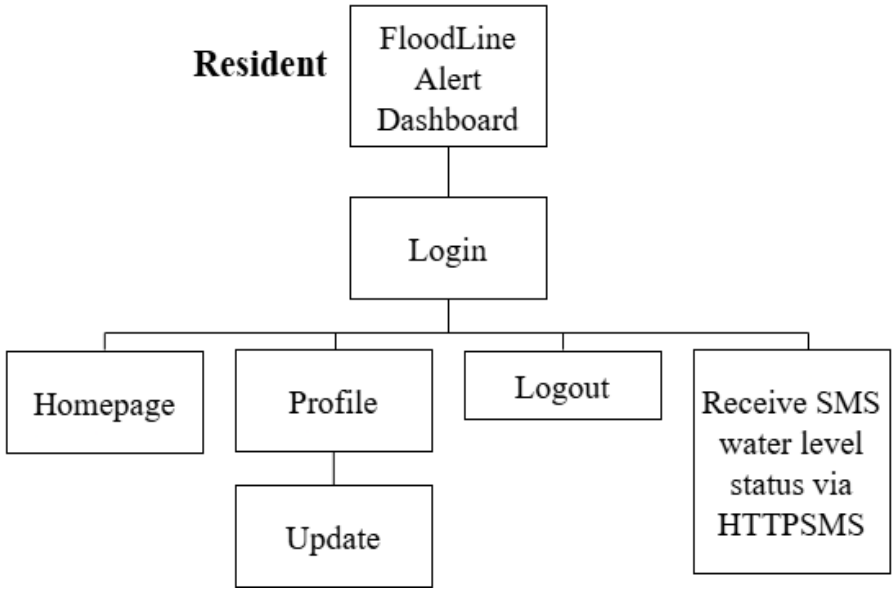


Figure 10.2 Structured Diagram (Resident)

The Figure 10.2 shows the Residents after logging in their verified account they can check the homepage or update their profile in addition they can receive the SMS Notification on an ongoing flooding.

This structure ensures each system segment has clearly defined responsibilities, allowing efficient information flow and reliable alerting.

Flowchart

The flowchart is a diagram that shows how the system works from start to finish. It starts with the sensors taking measurements, then shows what happens to that data as it goes through different steps.

Each part of the process relates to arrows so you can see the order things happen in. It also shows what choices the system makes along the way and what it does if something goes wrong.

The last part shows how the system sends a message to the user and gets feedback, so the process can keep improving.

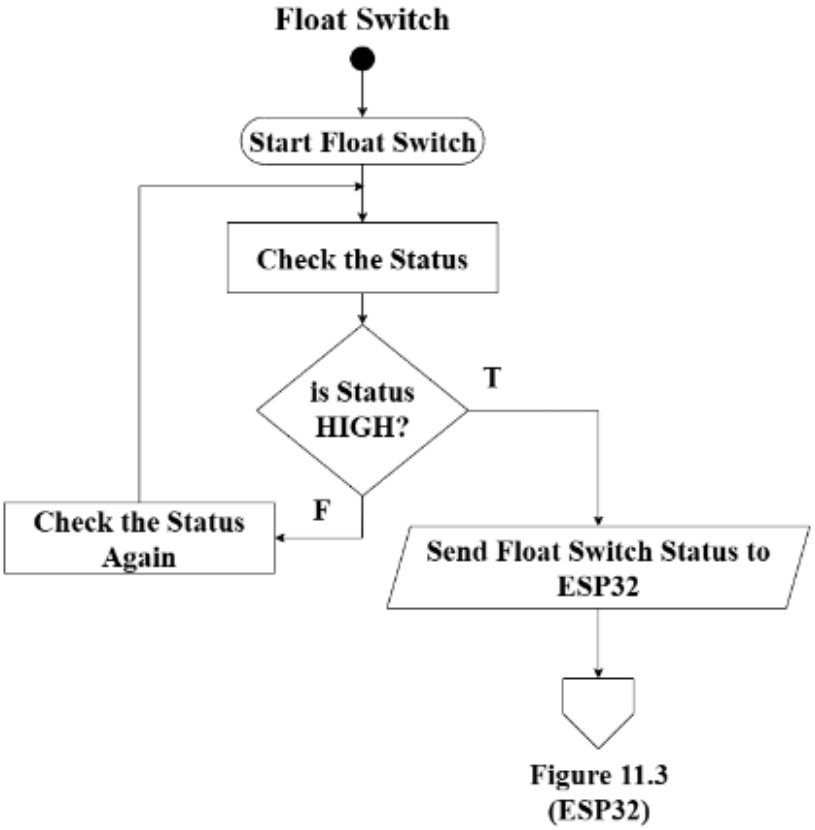


Figure 11.1 System Flowchart (Float Switch)

The figure 11.1 illustrates how a float switch functions as a crucial part of a water level monitoring system. The process begins when the float switch is activated, starting its task of continuously checking the status of the water level. The float switch keeps an eye on whether the water reaches a specific height, which is marked as the "HIGH" status. If the float switch detects that the water level is indeed high, it immediately sends this important status information to the ESP32 microcontroller, a small but powerful device that processes the data and makes decisions based on it. This action triggers the next steps in the monitoring system workflow, ensuring appropriate responses like alerts or further data recording. If the water level is not high, the float switch goes back to simply monitoring and waits for the next check, repeating this cycle continuously to keep tabs on water levels. This process is simple

yet vital because it provides timely and accurate updates on water levels, which helps prevent flooding or other water-related issues by informing the rest of the system when action is needed.

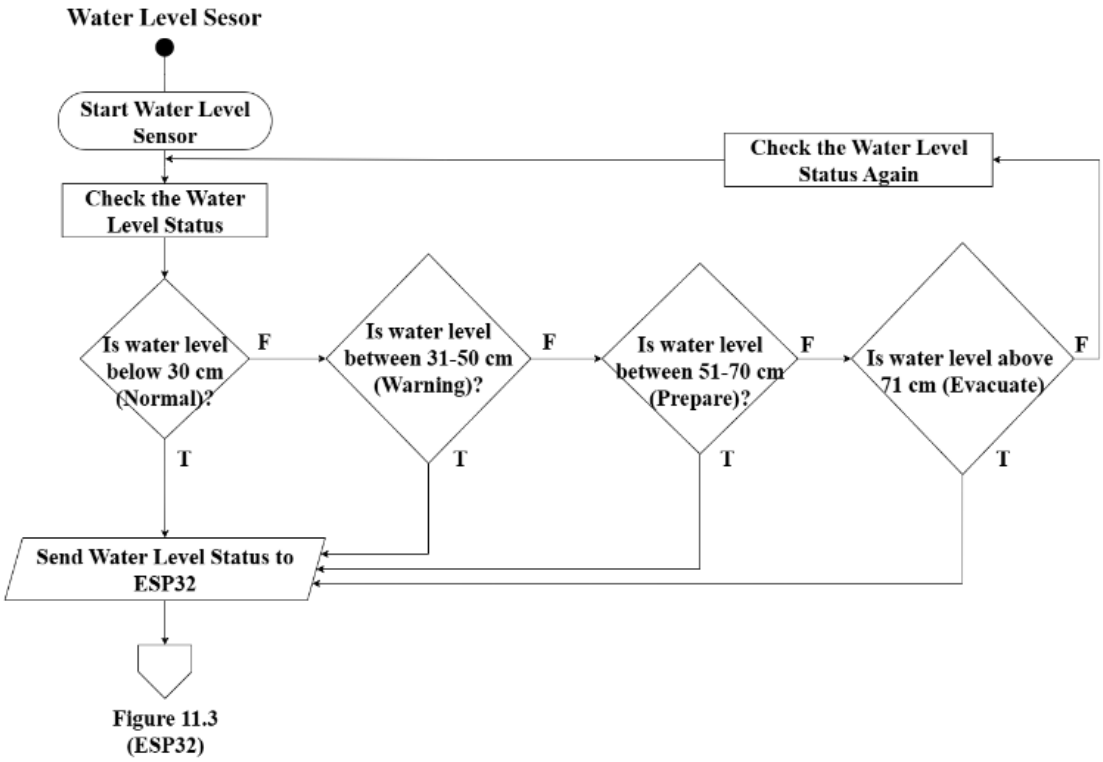


Figure 11.2 System Flowchart (Water Level Sensor)

The figure 11.2 illustrates how the water level sensor works to monitor the height of water and classify it into different levels to guide appropriate actions. It starts when the Water Level Sensor activates and begins checking the water level status. The water level is first checked to see if it is below 30 cm, which is considered normal. If so, the sensor sends a "normal" status to the ESP32 for regular monitoring. If the water level is not normal, the process checks if the water is between 31 to 50 cm, which is a warning level indicating that water is rising, and caution is needed. If it's not in this range, the sensor checks if the water level is between 51 and 70 cm, signaling a prepare level where people start readying for possible evacuation or safety measures. Finally, if the water level exceeds 71 cm, it reaches an evacuation level, indicating a critical situation where residents should evacuate for safety. Each time the sensor identifies a

specific water level range, it sends that status to the ESP32 microcontroller for further processing and response. This flow ensures a continuous watch over changing water levels and helps provide timely, categorized alerts to keep the community safe and informed.

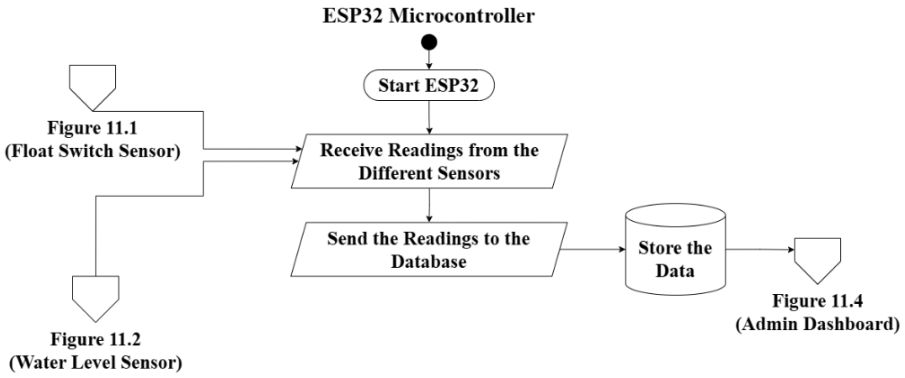


Figure 11.3 System Flowchart (ESP32 Microcontroller)

The figure 11.3 illustrates the process of how the ESP32 Microcontroller operates with sensor data and database storage. The system begins when the ESP32 collects readings from connected sensors, such as water level and float switch, and sends this data to the database for storage.

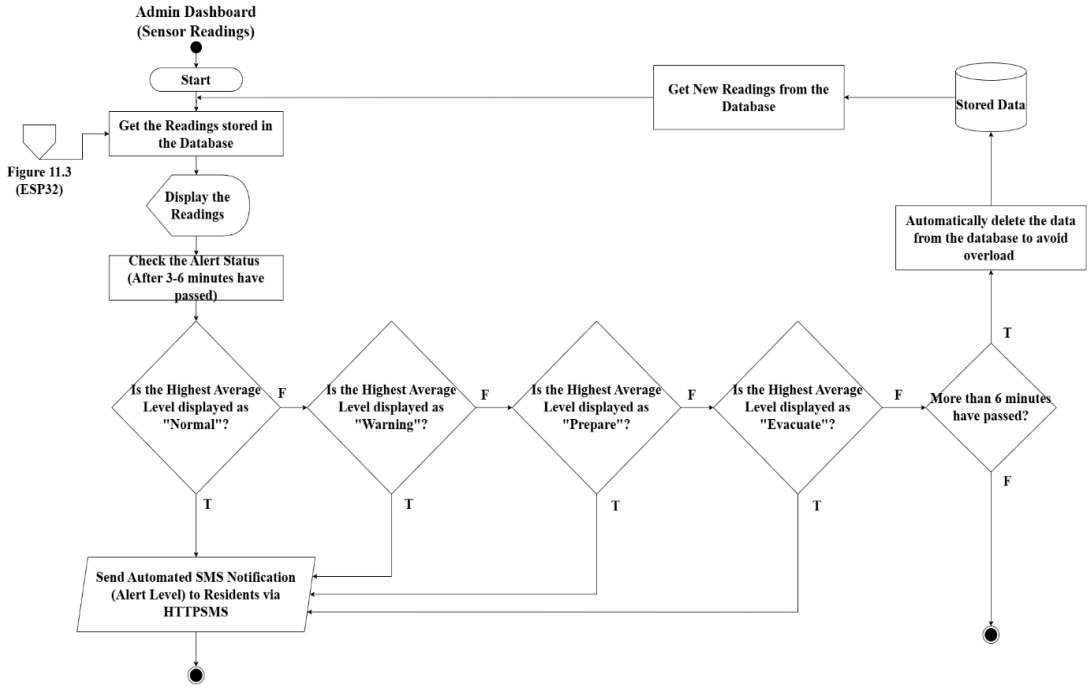


Figure 11.4 System Flowchart (Admin Dashboard)

The figure 11.4 illustrates the process of monitoring and alert notification in the Floodline Alert System (Admin Dashboard - Sensor Readings). The system begins by retrieving sensor readings stored in the database and displaying them for monitoring. After three-six minutes, the system checks the highest average level of the readings to determine the alert status, which may be categorized as Normal, Warning, Prepare, or Evacuate. If the alert level requires action, the admin has the option to send automated SMS notification (alert level) to the residents via HTTPSMS. The system then continuously retrieves new readings, while older data is automatically deleted after six minutes to prevent database overload. This cycle ensures real-time monitoring, accurate alert assessment, and timely notifications to residents during flood events.

Component List

A detailed list of all components is essential for successful assembly, troubleshooting, and future maintenance. This table provides component names, quantities, and descriptions for each item used in the Floodline Alert System.

Qty	Component	Description
1	ESP32 Dev Board	Main microcontroller (handles sensors, logic, communications; 3.3V logic)
1	HTTPSMS Gateway	An application software which will sends automated SMS alerts to registered mobile numbers.
1	Water Level Sensor – Analog pin	Water level sensor (3.3V, requires voltage, and output (Analog pin – S) to ESP32 via GPIO32 pin)

1	INA219 Current/Voltage Sensor	Optional, for monitoring system power consumption and battery voltage
1	PowMr 20W Solar Panel	Main renewable power supply
1	12V 7Ah/9Ah VRLA or 4x 32650 LiFePO4	Rechargeable battery for continuous operation
1	PowMr 30A PWM Charge Controller	Regulates solar charging and protects battery
1	LM2596 DC-DC Buck Module	Converts battery voltage to stable 5V for ESP32 and GSM module
1	1000µF Electrolytic Capacitor	Placed near GSM module Vcc for power smoothing
2	Resistor, 20kΩ (voltage divider)	For 3.3V-safe logic level at ESP32 inputs
1	Common Ground Bus or Rail	Ensures all modules share a common, reliable ground
1	IP65 Waterproof Box/Enclosure	Protect electronics outdoors
1	Breadboard or Custom PCB	Assembly platform: PCB recommended for field reliability
-	Jumper Wires, Connectors	For all signal and power wiring

Table 12. Component List

The component list details every item required to assemble, deploy, and maintain the system, specifying both function and usage. For instance, the ESP32 Dev Board serves as the main controller, directly connected to the water level sensor for

water level monitoring and the buck converter for current stability. The HTTPSMS gateway handles SMS communications. Power management is achieved with a PowMr 20W Solar Panel, a 12V 7Ah/9Ah VRLA battery or four 32650 LiFePO4 cells, and a PowMr 30A PWM Charge Controller; voltage is regulated by an LM2596 Buck Converter to meet the 3.3V/5V requirements of the electronics. Components like resistors for voltage dividers, capacitors for power smoothing, transistors for driving outputs, wiring, and an IP65 waterproof enclosure ensure system safety, reliability, and weather resistance.

Wiring Diagram

The wiring diagram is a graphical representation of all electronic connections, using standard symbols for clarity during assembly, testing, and troubleshooting.

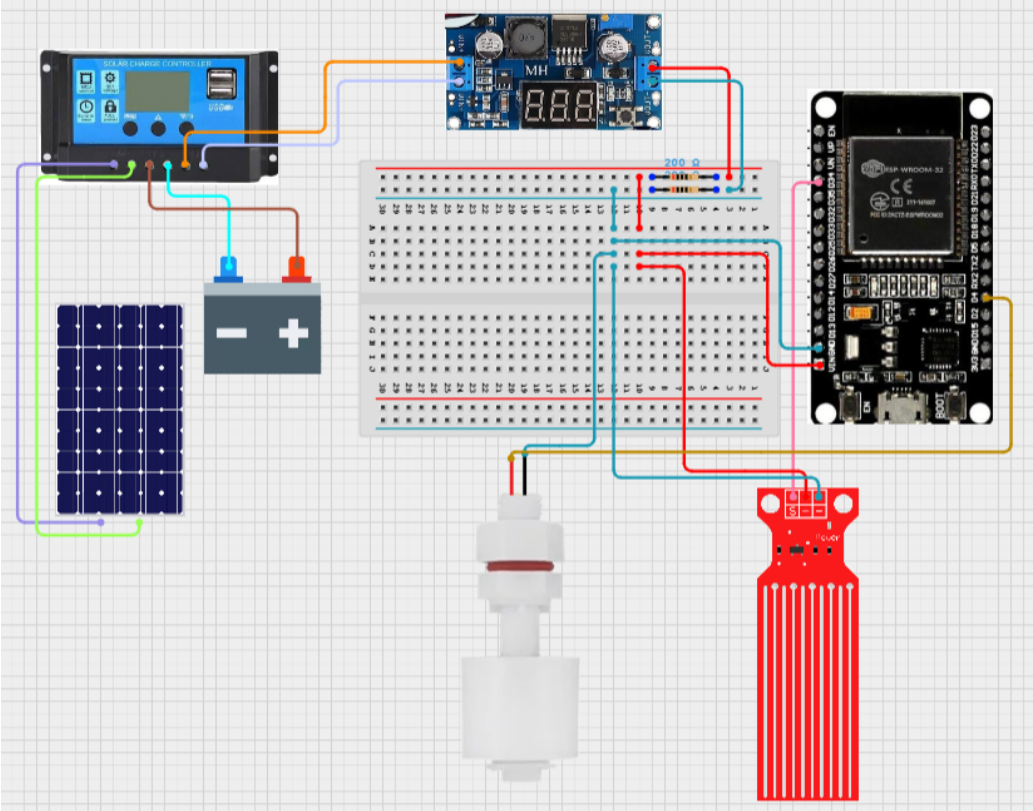


Image 13. Wiring Diagram

The Image 13 translates the schematic into universal electronic symbols and layout, enabling practical hardware assembly. It shows: the ESP32 microcontroller centrally, with arrows and lines indicating specific pin connections to (for example) the Water level sensor via GPIO34, GPIO4 for Float Switch. Power paths from solar panels and battery through the charge controller and buck converter are clearly indicated, with capacitors/resistors for stability. This diagram aids technicians and engineers in error-free assembly and maintenance.

Circuit Diagram

The circuit diagram translates the component list into an electrical design, specifying how each component connects to power, ground, control signals, and interfaces, ensuring reliable and safe operation.

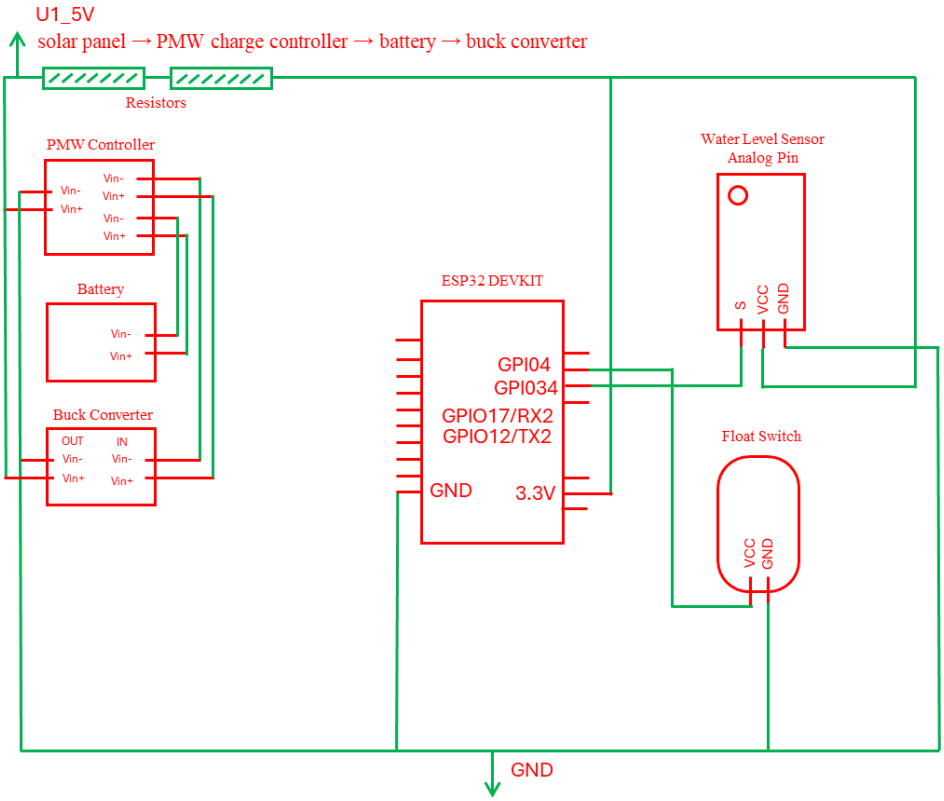


Figure 14. Circuit Diagram

The figure 14 provides a technical blueprint showing how each component connects electrically. The ESP32's GPIO pins are assigned as follows: GPIO34 for

water level sensor, GPIO4 for float switch. The DC-DC buck converter supplies a regulated 5V rail to both ESP32 and GSM modules, stabilized by a 1000µF electrolytic and 0.1µF ceramic capacitor/resistor for current surge prevention. The power system is integrated as: solar panel → charge controller → battery → buck converter → system, with all grounds joined on a central rail. This schematic guarantees that each functional block receives the correct supply, logic levels, and interconnections necessary for robust operation.

Deliver Phase

In this stage, the researchers presented the system architecture, showing how the sensors, ESP32 microcontroller, output devices, and software interfaces connect and work together.

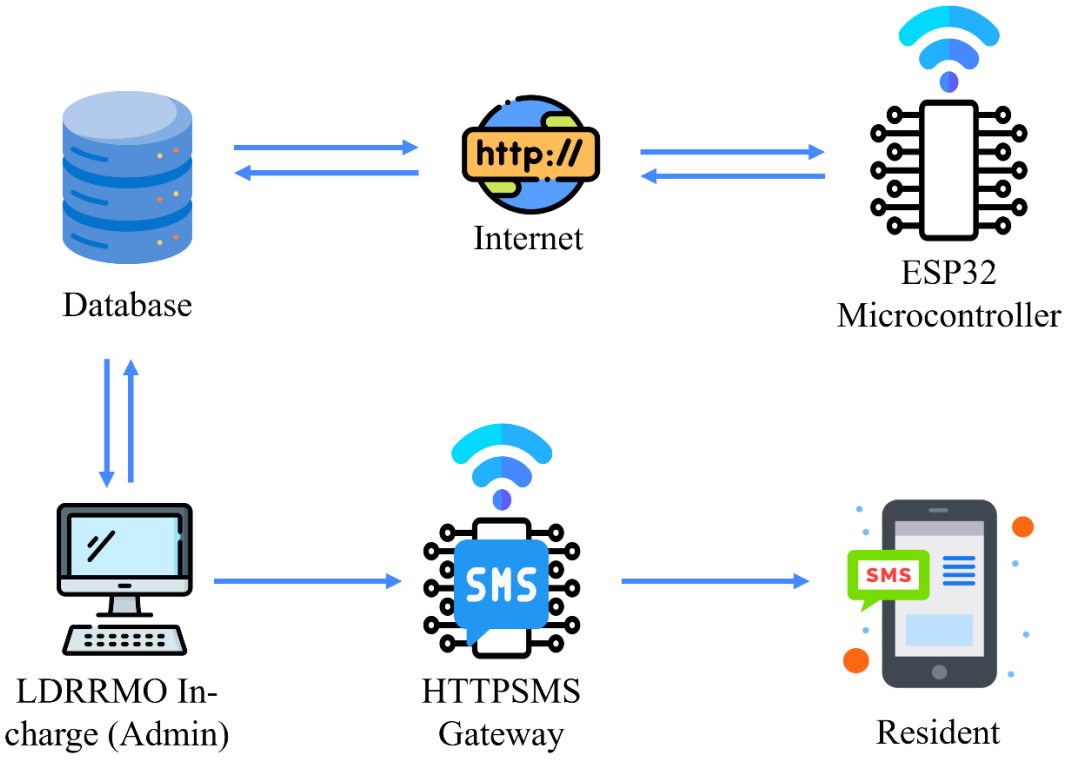


Figure 15. System Architecture Diagram

The figure 15 illustrates how the core components work together to deliver automated, real-time flood warnings to the community. The LDRRMO in-charge

(admin) manages resident information and system configurations through a computer interface linked to a centralized database, which stores user records, sensor data, and alert logs. The ESP32 microcontroller serves as the control hub, synchronizing with the database via internet to obtain the latest resident data, continuously monitoring connected flood sensors, and, upon detecting flood conditions, commanding the HTTPSMS gateway to send automated SMS alerts directly to residents' mobile phones. This architecture establishes a clear flow of information, admin inputs and monitoring feed into the database, the ESP32 retrieves and acts on this information, and residents receive timely SMS notifications ensuring centralized management, automated alerting, reliable communication even without internet access, and a scalable, modular design for future enhancements

CHAPTER IV

DEVELOPMENT AND TESTING

This chapter gives an overview of the study, including the programming language used and the different modules of the system. It also explains the testing done, such as alpha testing, beta testing, sensor initialization, boundary testing, load testing and stress testing. Finaly the chapter describes the plan for putting the system into action.

Description and Prototype

The study "Floodline Alert: Arduino-Based Flood Warning with SMS Notification" include various methods on developing the system. The system aims to address the problem faced by the Municipality of Guipos, Local Disaster Risk Reduction and Management Office (LDRRMO).

Months	**July**				**August**				**September**				**October**				**November**				**December**			
weeks	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
DISCOVER																								
Planning																								
Data Gathering																								
Requirements Analysis																								
DESIGN																								
GUI																								
DEVELOP																								
Coding																								
Testing																								
DELIVER																								
Implementation																								

Table 4.0 Gantt Chart

The table 4.0 is structured into four sequential phases: DISCOVER, DESIGN, DEVELOP, and DELIVER, running from the month of July through January. The project initiates with the Discover phase, focused on defining the project scope and requirements, and spanning from July to September. This phase starts with planning followed by the crucial data gathering, which is the longest task in this phase. This activity involves extensive research and interaction with key respondents, such as Engr.

Renan F. Mangle the Head of Office of the LDRRMO, to comprehensively document the specific needs of the Municipality of Guipos. The gathered information is then analyzed during the Requirements Analysis task to formalize the system specifications.

Following the documentation of requirements, the project moves into the DESIGN phase, which focuses on creating the system's architecture and user interface. This phase starts in September and runs until October, lasting a total of eight weeks. The primary activity is GUI Design, where the visual layout, usability, and overall look and feel of the system are drafted and finalized based on the outcomes of the Requirements Analysis. This task ensures the system is user-friendly and meets the functional needs of the end-users.

The DEVELOP phase is dedicated to building and testing the system, spanning from October to December. This is the most labor-intensive part of the project. It begins with coding, the longest single task in the entire project, which involves translating the design specifications into working software and hardware (ESP32) with its modules. Once the coding is underway, the Testing task begins, starting in the 2nd week of October and running concurrently. Testing involves rigorous quality assurance to identify and fix bugs, ensuring the system operates correctly and meets all defined requirements before delivery.

The final stage is the DELIVER phase, which focuses on evaluating the system's applicability and providing a final recommendation, spanning from December. This three-week phase is dedicated to System Recommendation. This task involves presenting the results of the data gathering, the performance of the developed prototype, and the overall technical feasibility and strategic viability of the system. The output is a formal report or presentation recommending whether the system should proceed to a full-scale implementation based on the collected data and analysis.

Development

The Floodline Alert: Arduino-Based Flood Warning with SMS Notification system relies on a combination of programming languages and technologies for its functionality.

PHP manages the system's backend, handling data processing, database communication, and core website functions, with HTML and CSS used for structuring and styling the appealing user interface. For the hardware component, C++ is used on the ESP32 module, utilizing libraries like WiFi.h, HTTPClient.h, and ArduinoJson.h to process sensor data and transmit it to the database. MySQL serves as the robust database management system, integrating seamlessly with PHP to store, organize, and quickly retrieve sensor readings and user records, ensuring administrative and resident updates are immediately reflected across the system. This modular design supports core functions like reading sensor data and sending SMS alerts.

Sensor Readings Module:

The C++ code, implemented on the ESP32 via the Arduino IDE, acts as the primary interface between the physical flood sensors and the digital Floodline Alert system. Its core responsibility is to periodically sample the environment, process the raw data using calibration and threshold logic, and securely transmit the crucial readings to the central database over Wi-Fi.

The C++ Code:

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>

// ======== CONFIG: change these ========
const char* WIFI_SSID = "Keynotes";
const char* WIFI_PASS = "walaypassword";

// Your PC running XAMPP (LAN IP), plain HTTP
const char* BASE_URL = "http://10.75.66.6/floodline/api"; // <-- current PC LAN IP

// Match an existing Sensor in your DB/Admin UI
const int SENSOR_ID = 2;
```

```
// Pins
const int PIN_WATER = 34;   // ADC1 channel (GPIO34)
const int PIN_FLOAT = 27;   // digital input with pullup

// Timing
const uint32_t SAMPLE_PERIOD_MS = 10 * 1000; // send reading every 10s

// ======== Calibration for analog -> centimeters ========
// Measure raw ADC at two known water levels and update these.
const int RAW_AT_0CM   = 400;   // raw ADC when water is 0 cm (dry baseline)
const int RAW_AT_MAXCM = 3000;  // raw ADC when water is MAX_CM fully covered
const float MAX_CM     = 100.0; // your reference max height (cm)
// ======================================================

// Map raw ADC to centimeters (linear calibration)
float levelCmFromRaw(int raw) {
  int r = constrain(raw, RAW_AT_0CM, RAW_AT_MAXCM);
  float pct = (float)(r - RAW_AT_0CM) / (RAW_AT_MAXCM - RAW_AT_0CM);
  pct = constrain(pct, 0.0f, 1.0f);
  return pct * MAX_CM;
}

// Determine status with float-switch gating for Evacuate
String statusFrom(float cm, bool floatActive) {
  String s;
  if (cm <= 30.0f)       s = "Normal";
  else if (cm <= 50.0f)  s = "Warning";
  else if (cm <= 70.0f)  s = "Prepare";
  else                   s = "Evacuate";

  if (s == "Evacuate" && !floatActive) {
    // Gate: only send Evacuate if float switch is activated
    s = "Prepare";
  }
  return s;
}

void sendReadingToServer(float cm, bool floatActive, const String& status) {
  if (WiFi.status() != WL_CONNECTED) return;
  HTTPClient http;
  String url = String(BASE_URL) + "/readings.php?action=add";
  StaticJsonDocument<256> doc;
  doc["sensor_id"]    = SENSOR_ID;
  doc["level_cm"]     = cm;
  doc["float_switch"] = floatActive ? 1 : 0;
  doc["status"]       = status;
  String payload;
  serializeJson(doc, payload);

  http.begin(url);
  http.addHeader("Content-Type", "application/json");
  http.setTimeout(5000);
  int code = http.POST(payload);
  String resp = http.getString();
  http.end();

  Serial.printf("[POST readings] code=%d, resp=%s\n", code, resp.c_str());
}

void connectWifi() {
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  Serial.print("WiFi connecting");
  uint8_t tries = 0;
  while (WiFi.status() != WL_CONNECTED && tries < 40) {
    delay(500); Serial.print(".");
    tries++;
  }
  Serial.println(WiFi.status() == WL_CONNECTED ? " OK" : " FAIL");
}

void setup() {
  Serial.begin(115200);
  pinMode(PIN_FLOAT, INPUT_PULLUP);
  connectWifi();
  Serial.print("WiFi IP: ");
```

```
  Serial.println(WiFi.localIP());
  Serial.printf("BASE_URL=%s SENSOR_ID=%d\n", BASE_URL, SENSOR_ID);
}

void loop() {
  static uint32_t lastSample = 0;
  uint32_t now = millis();

  // Periodic sensor reading -> DB
  if ((now - lastSample >= SAMPLE_PERIOD_MS)) {
    lastSample = now;
    int raw = analogRead(PIN_WATER);
    bool floatActive = (digitalRead(PIN_FLOAT) == LOW); // switch closed -> LOW
    float cm = levelCmFromRaw(raw);
    String status = statusFrom(cm, floatActive);
    sendReadingToServer(cm, floatActive, status);
    Serial.printf("[reading] raw=%d, cm=%.1f, float=%d, status=%s\n",
                  raw, cm, floatActive, status.c_str());
  }
}
```

Status Determination and Safety Gating (statusFrom)

The **statusFrom** function is the intellectual core of the flood logic, converting calibrated measurements into actionable alert statuses.

```
// Determine status with float-switch gating for Evacuate
String statusFrom(float cm, bool floatActive) {
  String s;
  if (cm <= 30.0f)       s = "Normal";
  else if (cm <= 50.0f)  s = "Warning";
  else if (cm <= 70.0f)  s = "Prepare";
  else                   s = "Evacuate";

  if (s == "Evacuate" && !floatActive) {
    // Gate: only send Evacuate if float switch is activated
    s = "Prepare";
  }
  return s;
}
```

Primary Thresholds: The initial if/else if block assigns a status based on linear thresholds defined in centimeters (cm). This is standard alert progression logic.

- **Normal** (≤ 30.0 cm), **Warning** (≤ 50.0 cm), **Prepare** (≤ 70.0 cm), and **Evacuate** (> 70.0 cm).

The Critical Gating Line: The line if (s == "Evacuate" && !floatActive) implements a **failsafe redundancy**.

- It checks if the calculated status is "Evacuate" *and* if the digital float switch is *not* physically active (!floatActive).

- If the high-water level is detected by the analog sensor but the mechanical float switch has *not* been triggered, the status is immediately demoted to **"Prepare"**.
- This prevents false "Evacuate" alerts that could be caused by noise, temporary splashing, or a faulty analog sensor, requiring both systems to agree on the highest critical status.

Data Transmission and Integration (sendReadingToServer)

The **sendReadingToServer** function is responsible for packaging the data and ensuring its secure delivery to the web server's API, facilitating the integration between the hardware (ESP32) and the software (PHP/Database).

```
void sendReadingToServer(float cm, bool floatActive, const String&
status) {
  // ... (omitted: WiFi check and HTTPClient setup)

  StaticJsonDocument<256> doc;
  doc["sensor_id"]    = SENSOR_ID;
  doc["level_cm"]     = cm;
  doc["float_switch"] = floatActive ? 1 : 0;
  doc["status"]       = status;
  String payload;
  serializeJson(doc, payload);

  http.begin(url);
  http.addHeader("Content-Type", "application/json");
  http.setTimeout(5000);
  int code = http.POST(payload);
  // ... (omitted: response handling)
}
```

JSON Packaging: The use of StaticJsonDocument is crucial for creating the **JSON payload**. This ensures the data is transmitted in a standard, structured format ({"sensor_id": 2, "level_cm": 72.1, "float_switch": 1, "status": "Evacuate"}).

- The line doc["float_switch"] = floatActive ? 1 : 0; converts the C++ boolean (true or false) into a simple integer (1 or 0), which is the standard format for boolean values in most SQL databases.

HTTP POST Request: The lines http.addHeader("Content-Type", "application/json"); and int code = http.POST(payload); define the transmission protocol.

- The Content-Type header tells the receiving PHP server that the data being sent is in JSON format, allowing the backend script to parse it correctly.
- The http.POST(payload) command executes the request, sending the structured sensor data to the specified API endpoint (http://10.75.66.6/floodline/api/readings.php?action=add), which then inserts the reading into the database for the administrator to view on the website.

Main Loop and Timing (loop)

The **loop()** function manages the continuous, periodic operation of the device:

```
void loop() {
  static uint32_t lastSample = 0;
  uint32_t now = millis();

  // Periodic sensor reading -> DB
  if ((now - lastSample >= SAMPLE_PERIOD_MS)) {
    // ... (code to read sensors and send data)
  }
}
```

Non-Blocking Delay: The code uses the pattern if ((now - lastSample >= SAMPLE_PERIOD_MS)) instead of the simple delay() function. This ensures the 10-second sampling period is non-blocking. This is critical in embedded systems as it allows the ESP32 to continue handling background Wi-Fi management and other tasks while waiting for the next reading interval.

In summary the ESP32 C++ (Sensor Readings Module) code functions as the **hardware interface** and **data processing core** of the Floodline Alert System. It is responsible for continuously sampling the water level and float switch sensors every 10 seconds. The code performs **linear calibration** to convert raw electrical readings into

accurate water depth in centimeters (cm) and then determines the appropriate alert status ("Normal," "Warning," "Prepare," or "Evacuate") based on predefined thresholds.

Crucially, the system employs a **safety gating mechanism** in the statusFrom function: the critical "Evacuate" status is only confirmed if the high-water level is met and the physical float switch is simultaneously triggered, preventing false alarms.

Finally, the code packages the processed data (ID, cm level, float status, and final alert status) into a **JSON payload** and securely transmits it to the central PHP API via an **HTTP POST request**, making the real-time information immediately available to the web administrator interface.

SMS Alert Module

The system's core functionality for sending real-time flood warnings is managed by the integration of PHP, a custom API built within the system, and the external HTTPSMS Gateway. This combination is critical for transforming an administrative decision into a delivered message.

Configuration

This file defines the settings for the SMS gateway, ensuring all messages are sent through the designated service provider.

```
// Default provider for sending SMS
'provider' => 'httpsms',
// Endpoint: https://api.httpsms.com/v1/messages/send
'httpsms_api_url' => 'https://api.httpsms.com/v1/messages/send',
'httpsms_api_key'                                                    =>
'uk_w2adRlCdJPjmFj_nLMEu0cEUbp16RHeHHEuu7J4BLXLE14dz4tbxi_kDvAmP_uf4',
```

- **Provider**: 'httpsms' is set as the default provider, confirming the system relies on an external HTTP SMS API for message delivery.

- **API Credentials**:

    - 'httpsms_api_url': Specifies the exact endpoint for message transmission.

- 'httpsms_api_key': Contains the unique key (uk_w2adRlCdJPjmFj_nLMEu0cEUbp16RHeHHEuu7J4BLXLE14dz4tbxi_kDvAmP_uf4) required for authentication with the service.
- 'httpsms_from': Defines the official sender number (+639940996035).

- **Number Format**: 'httpsms_number_format' => 'e164' ensures recipient numbers are formatted correctly (e.g., +639XXXXXXXXX) as required by the international standard before sending.

Sensor Data Ingestion and Auto-Pruning Logic

This file is the direct communication endpoint for the ESP32 hardware. It processes the JSON data sent from the microcontrollers and manages the continuous storage and cleanup of sensor readings.

A. Data Ingestion (action=add)

- **Endpoint Function**: This script accepts the JSON payload from the ESP32 via HTTP POST ($action === 'add').
- **Data Fields**: It captures the sensor_id, status (e.g., "Evacuate"), levelCm (water level), and floatSw (float switch state).
- **Table Management**: It ensures the SensorReading table exists and dynamically adds new columns (LevelCM and FloatSwitch) if they are missing, ensuring the database schema matches the ESP32's output.
- **Data Insertion**: It inserts the validated reading into the SensorReading table.

B. Auto-Pruning and Auto-Evacuate Logic (Critical Functionality)

After successfully inserting a new reading, the script immediately executes two critical functions:

Auto-Pruning:

```
$conn->query("DELETE FROM SensorReading WHERE CreatedAt < (NOW() - INTERVAL 6 MINUTE)");
```

This line is a core maintenance function. It automatically deletes all sensor readings older than 6 minutes from the database. This is essential for maintaining system performance and stability under a "Stress Test" scenario, preventing database overload from continuous 10-second readings (as noted in Test Case 005 and 004 discussions).

Autonomous "Evacuate" Trigger (Gating Logic):

This block implements the automatic alert dispatch logic (related to Test Case 003, Step 2, and the system's failsafe redundancy).

- Averaging Status: It retrieves all sensor readings for the last 6 minutes and calculates an average status value by mapping status names ("Normal"=0, "Evacuate"=3) to numeric scores.
- Evacuation Threshold:

```
if($avg >= 2.5){ // 2.5 is the threshold for an average status between 'Prepare' (2) and 'Evacuate' (3) // ...
```

If the average status remains at or above 2.5 for 6 minutes (meaning the flood condition is consistently high), the script proceeds to trigger the alert.

SMS Dispatch

If the calculated threshold is met **and** no "Evacuate" alert has been queued in the last 6 minutes ($exists = false), the script *automatically* inserts an **"Evacuate"** alert into the AlertQueue and **immediately sends the corresponding SMS to all registered recipients** without requiring administrator intervention. This acts as the safety net for critical situations.

Alert Dispatch and Logging

This handles the administrator's manual alert actions and logs every delivery attempt.

```
/ Get recipients for this sensor
$stmtR = $mysqli->prepare("SELECT u.ContactNumber FROM
SensorRecipient sr INNER JOIN User u ON sr.UserID =
u.UserID WHERE sr.SensorID = ? ...");
// ...
// Build message if not provided using templates
// ...
// Send immediately via provider
$results = sms_send($recipients, $message);
// Log detailed delivery results for every recipient
$stmtD = $mysqli->prepare("INSERT INTO AlertDelivery
(AlertID, SensorID, Recipient, Provider, HttpCode, Sent,
Response, MessageID, Status, Network) VALUES
(?,?,?,?,?,?,?,?,?,?)");
```

SMS Queueing and Dispatch Logic

This file manages the communication workflow between the Admin UI and the SMS service, handling both manual and status-check requests.

A. Manual/Immediate SMS Dispatch (action=send)

This is the primary function triggered by the Admin UI to send alerts, either manually or in response to a prompt.

- **Queue Insertion**: It first inserts the alert details (SensorID, Status, Message) into the AlertQueue table, marking it as Processed = 0 (unprocessed).
- **Recipient Retrieval**: It queries the database to gather all ContactNumbers associated with the specific SensorID from the SensorRecipient and User tables.
- **Message Templating**: If the administrator does not provide a custom message, it uses predefined templates for "Normal," "Warning," "Prepare," and "Evacuate" statuses.
- **Immediate Dispatch**: It calls the sms_send function, utilizing the provider configured in sms_config.php.

- **Delivery Logging**: It logs the results of *every* recipient's delivery (success/fail, HTTP code, Message ID, network) into the AlertDelivery table, ensuring a complete audit trail (related to Test Case 008, Data Logging).
- **Status Update**: It immediately updates the AlertQueue record, setting Processed = 1, completing the queue transaction.

B. Status Check (action=pending)

- **Purpose**: This function is used by the Admin UI (the website) to check if an alert has been created but not yet manually sent by the administrator.
- **Query**: It retrieves the latest unprocessed alert (Processed = 0) from the AlertQueue for the requested sensor. The Admin UI would then display this pending alert to the operator for action.

C. Acknowledge (action=ack)

- **Purpose**: This function allows the Admin UI to mark a specific pending alert (Processed = 0) as processed (Processed = 1) without sending an SMS. This would be used if the administrator decides the alert is no longer necessary or has been handled by other means.

In summary, the PHP backend forms the **central nervous system** of the Floodline Alert system, executing all data management and communication logic. It leverages **configuration** to receive JSON data streams from the ESP32, immediately inserts new readings, and performs a crucial **auto-pruning routine** to delete data older than 6 minutes, ensuring database stability under continuous sensor load.

Most critically, it implements the **failsafe autonomous alert logic**: if the average flood status remains high (above the 2.5 threshold) for a sustained period, the system automatically bypasses manual intervention and dispatches an **"Evacuate" SMS alert** via the configured HTTP SMS service. The script handles the manual alert

initiation by the administrator, retrieves recipient lists, applies message templates, and rigorously logs every SMS delivery attempt into the database for a complete, auditable record.

Testing

System testing is a crucial step that verifies the entire project meets its goals and is free of major defects. It is essential for ensuring functionality and reducing operational risks.

For the Floodline Alert System, validation was comprehensive, covering both the software (website and logic) and the hardware (ESP32, sensors, and power). Specific hardware checks like Sensor Initialization, Power Supply, and Safety Testing were performed to confirm the physical components integrate correctly, perform reliably under stress, and handle failures safely. This approach confirms that both the digital and physical aspects of the system are robust and prepared for continuous, stable operation. The Test Case paraphernalia's is located on the appendix for further references.

Software (Website) Development

Website Role in System Control and Performance

The Floodline Alert System website functions as the critical command and performance monitoring center, moving beyond simple data display to enforce operational protocols. The system is designed with dual-mode alerting: it first presents critical "Evacuate" status to the administrator (See Appendix A: TC 003, Step 1) for immediate manual review, but it also features a crucial automation fallback (See Appendix A: TC 003, Step 2) where the SMS alert is automatically dispatched after a

set duration (6 minutes) to ensure residents are notified even if the administrator is delayed.

This architecture, verified by the Sensor Readings Response test, underscores the balance between human oversight and guaranteed automation. Furthermore, comprehensive non-functional tests. Load Testing, Stress Testing (See Appendix A: TC 005), and Compatibility Testing, confirm the website's stability and durability under continuous operation and high data volume, ensuring the critical command interface remains responsive and accessible across various devices and network loads.

Data Management, Integration, and Safety

A key function of the Floodline website is its role in data management and system integration.

The Integration Test (See Appendix A: TC 004) confirms the software correctly receives and processes the hardware data, performing the necessary logic (e.g., calculating average highest water levels and deleting old records to prevent overload, Appendix A: TC 004, Step 2) before displaying the official status to the administrator. This ensures data integrity and a smooth operational workflow. Additionally, the Data Logging test (TC 008) validates the website’s backend ability to serve as a reliable repository, capturing all water level events and SMS alert actions with precise timestamps, which is vital for post-flood audit and evidence.

Finally, the Safety Test (See Appendix A: TC 007) and Usability Test ensure the website environment is robust, displaying appropriate error messages (like "Sensor Disconnected,", See Appendix A: TC 007, Step 1) for abnormal conditions and remaining intuitive enough for the administrator to operate effectively after initial training.

ESP32 Development

The testing phase utilized a set of eight distinct test cases to validate the core functionalities, reliability, performance, and safety of the Floodline Alert System, encompassing both the ESP32 hardware development and its integration with the web backend. The results, detailed in Appendix A, consistently demonstrate that the system meets all specified operational requirements.

Hardware Initialization and Sensor Validation

These initial tests confirmed that the physical components were correctly wired, initialized, and capable of generating accurate, stable data for the system.

Test Case #: 001 - Sensor Initialization

- **Purpose:** To verify that the ESP32 module powers on correctly, establishes communication with the monitoring dashboard, and initializes the physical water level and float switch sensors.
- **Methodology:** The device was powered on, and the dashboard was monitored for device status. Serial output and dashboard readings were checked for initial data acquisition.
- **Result & Conclusion:** The test **passed**. The ESP32's indicator light confirmed power-on, and the device successfully appeared 'Online' on the dashboard, confirming basic connectivity and readiness for data transmission. (See Appendix A: Test Case #: 001)

The system confirmed successful power-on and network connectivity, ensuring the ESP32 is ready to sample and transmit data upon deployment.

Test Case #: 002 - Accuracy and Stability of Real-time Water Level Sensor and Float Switch

- **Purpose:** To validate the accuracy and stability of the sensors and verify that the calculated water level corresponds correctly to the four distinct alert status thresholds ("Normal" to "Evacuate").

- **Methodology:** Controlled amounts of water were introduced to the sensor setup to simulate water levels corresponding to 30.0 cm (Normal), 50.0 cm (Warning), 70.0 cm (Prepare), and >70.0 cm (Evacuate, requiring float switch activation). Real-time readings were observed for stability.

- **Result & Conclusion:** The test **passed**. Sensor readings stabilized quickly at each level, and the system correctly transitioned the displayed status (e.g., from Warning to Prepare), confirming the accuracy of the hardware-side calibration and threshold logic. (See Appendix A: Test Case #: 002)

The test validated the hardware-side calibration and flood logic, confirming that raw sensor data is reliably converted into accurate centimeter readings and correctly maps to the four defined alert statuses.

System Integration and Alert Logic Validation

These tests focused on the communication between the ESP32 and the web application (backend/database) and validated the system's complex alert queuing and decision-making logic.

Test Case #: 003 - Sensor Readings Response

- **Purpose:** To verify the system's two-tiered alert mechanism: initial manual approval for critical alerts and the failsafe automatic dispatch after a time delay.

- **Methodology:** An "Evacuate" status was triggered. The system was first checked to ensure it correctly prompted the administrator for manual action. The high status was then maintained for over 6 minutes to confirm the automatic SMS dispatch functionality.

- **Result & Conclusion:** The test **passed**. The dashboard correctly showed the "Evacuate" status and waited for administrator input (manual mode). After 6 minutes, the system autonomously sent the SMS alert to recipients, validating the critical failsafe logic required to guarantee timely notification. This automatic dispatch is handled by the data processing logic within the PHP readings.php script. (See Appendix A: Test Case #: 003)

The alert system successfully executed its failsafe protocol, transitioning from administrator review to guaranteed automatic SMS dispatch after the 6-minute sustained critical threshold was met.

Test Case #: 004 - Integration: Sensor + HTTPSMS gateway

- **Purpose:** To confirm end-to-end data flow, from the ESP32 data packet ingestion to the final database transaction, ensuring the integrity of the integrated system.

- **Methodology:** A full alert sequence was triggered twice. The system's response was observed for consistency, and the database was checked to ensure data processing (e.g., status averaging and deletion of old records) was executed correctly after new data arrival.

- **Result & Conclusion:** The test **passed**. The system responded consistently in both runs, and the PHP backend confirmed that data was correctly processed

and displayed, validating the reliable integrity of the hardware-software communication via the API gateway. (See Appendix A: Test Case #: 004)

End-to-end data flow was verified, confirming the PHP backend correctly ingests the ESP32's JSON payload, processes the data, and successfully communicates with the external SMS gateway.

Test Case #: 008 - Data Logging / Output

- **Purpose:** To verify that every critical event (sensor reading, status change, and SMS dispatch) is accurately recorded with a timestamp, ensuring data traceability and a reliable audit trail.
- **Methodology:** Multiple status changes and alert dispatches were executed. The database logs (AlertDelivery and SensorReading tables) were then retrieved and examined to ensure data was complete, structured, and accurately timestamped.
- **Result & Conclusion:** The test **passed**. All events were logged correctly and were retrievable in a structured format, fulfilling the requirement for system auditability, as implemented by the logging logic in the PHP alerts.php and readings.php files. (See Appendix A: Test Case #: 008)

The database successfully logged all sensor readings, status transitions, and every individual SMS delivery attempt, providing a complete, auditable record for post-event analysis.

Performance, Stability, and Safety Validation

These non-functional tests ensured the system could withstand operational stress and handle abnormal power and safety conditions gracefully.

Test Case #: 005 - Stress Test

- **Purpose:** To determine the system's stability and ability to handle continuous data processing and communications over an extended period.

- **Methodology:** The complete system (ESP32 constantly sending data, server processing) was run continuously for a minimum of 24 hours. System logs and physical component temperatures were monitored for crashes, data loss, or overheating.

- **Result & Conclusion:** The test **passed**. The system remained operational for the entire 24-hour period without crashing or overheating, confirming its reliability for continuous, long-term deployment. This is supported by the PHP backend's auto-pruning logic which prevents database overload. (See Appendix A: Test Case #: 005)

The system demonstrated high stability, maintaining full functionality for 24 hours under continuous data load without crashing, confirming deployment readiness.

Test Case #: 006 - Power Supply Check

- **Purpose:** To ensure the stability of the power source and verify the system's ability to recover correctly following an unexpected power interruption.

- **Methodology:** The main power supply voltage was measured to confirm it was within the acceptable range (5V. Power was then manually disconnected and immediately restored to test reinitialization.

- **Result & Conclusion:** The test **passed**. The voltage reading was stable at 5.01V. and the device successfully restarted and reconnected to the network and dashboard without requiring manual intervention, validating its robustness against power fluctuations. (See Appendix A: Test Case #: 006)

Power stability was confirmed, and the system demonstrated successful self-recovery and automatic reconnection after an abrupt power failure.

Test Case #: 007 - Safety Test

- **Purpose:** To verify that the system can safely handle abnormal conditions such as sensor faults or simulated electrical issues without causing permanent damage or crashing.
- **Methodology:** A sensor cable was manually disconnected to simulate a failure. Additionally, a non-critical short circuit was simulated on the ESP32. Recovery and fault reporting were checked.
- **Result & Conclusion:** The test **passed**. Upon sensor disconnection, the dashboard immediately displayed a "Sensor Disconnected" error, and the system did not suffer permanent damage from the short circuit, confirming its internal fault tolerance and safety design. (See Appendix A: Test Case #: 007)

The system proved safe and fault-tolerant, correctly reporting sensor disconnections and surviving simulated electrical faults without permanent damage.

Implementation Plan

Strategy	Activities	Persons Involve	Duration
Approval of the system.	Acceptance of the Project during final defense.	Researchers, Panelist	5 days
Installation	System Hosting	Researchers	3-5 days
Training	Train the user of the system	Researchers, Users	1week

Table 5. Implementation Plan

Table 5.0 presents the study's implementation plan. The system will be approved during the final defense, where both the researchers and panelists will assess the project's readiness for deployment. This approval is anticipated to take 5 days. After approval, the researchers will proceed with system installation and hosting setup, expected to be completed within 3 days. Furthermore, the researchers will conduct a training session for users, lasting one week, to ensure they understand the system's features and have ample time for practice and to address any questions.

CHAPTER V

IMPLEMENTATION RESULT, DISCUSSION, CONCLUSION AND RECOMMENDATION

Implementation Result and Discussion

This chapter presents the results gathered after the implementation of the Floodline Alert: Arduino-Based Flood Warning with SMS Notification system in the community. It discusses how the prototype performed during actual operation, emphasizing the behavior of the dual-sensor unit, the ESP32 controller, the back-end alert logic, and the web dashboard throughout testing and deployment. The chapter also analyzes the strengths, limitations, and overall effectiveness of the Floodline Alert system in providing automated real-time water level monitoring, guaranteed alerts, and detailed logging to support local disaster risk reduction and management efforts. Finally, the conclusion highlights the key findings of the study, while the recommendations propose improvements and possible future enhancements for the Floodline Alert system.

The Floodline Alert: Arduino-Based Flood Warning with SMS Notification system was successfully implemented and rigorously tested to validate its core mandate: to provide timely, accurate flood warnings to residents through an automated SMS platform.

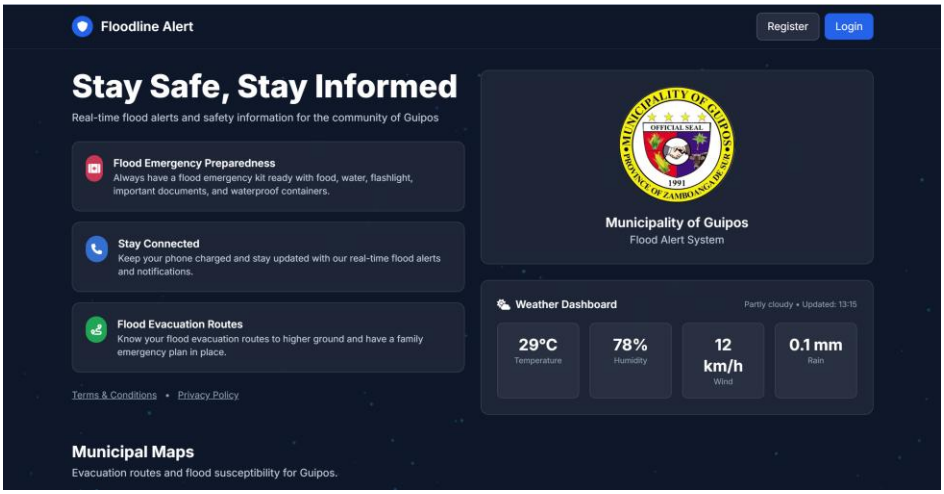


Image 1. System UI

The system comprises a Public UI for residents (Image 1. System UI) and a comprehensive Admin Panel for the Local Disaster Risk Reduction and Management Office (LDRRMO).

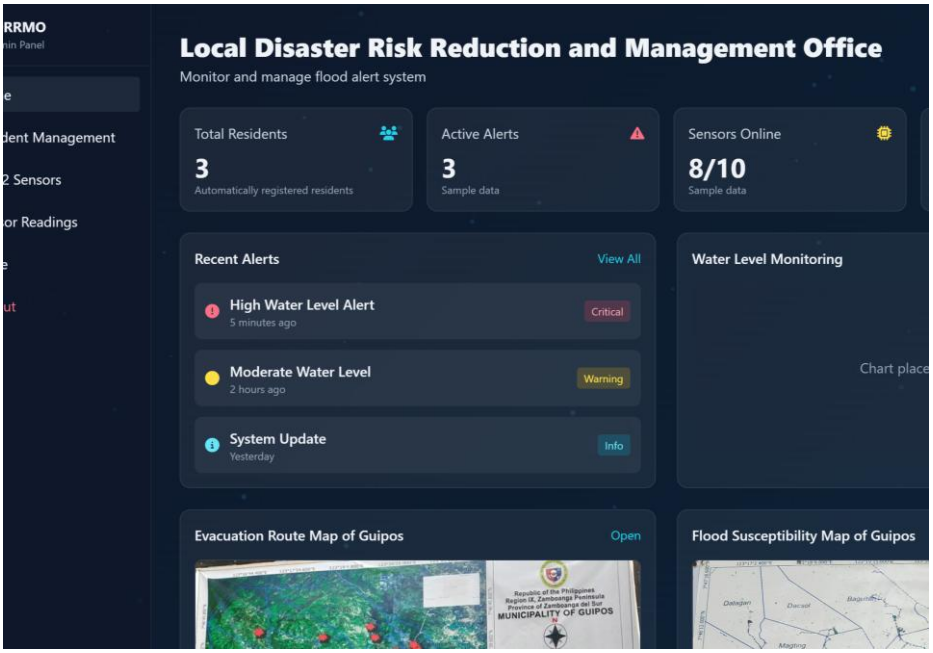


Image 2.0 Admin – Homepage

The Admin Homepage (Image 2.0 Admin – Homepage) serves as the control center, providing key performance metrics such as total registered residents, active alerts, and a snapshot of recent high-level incidents.

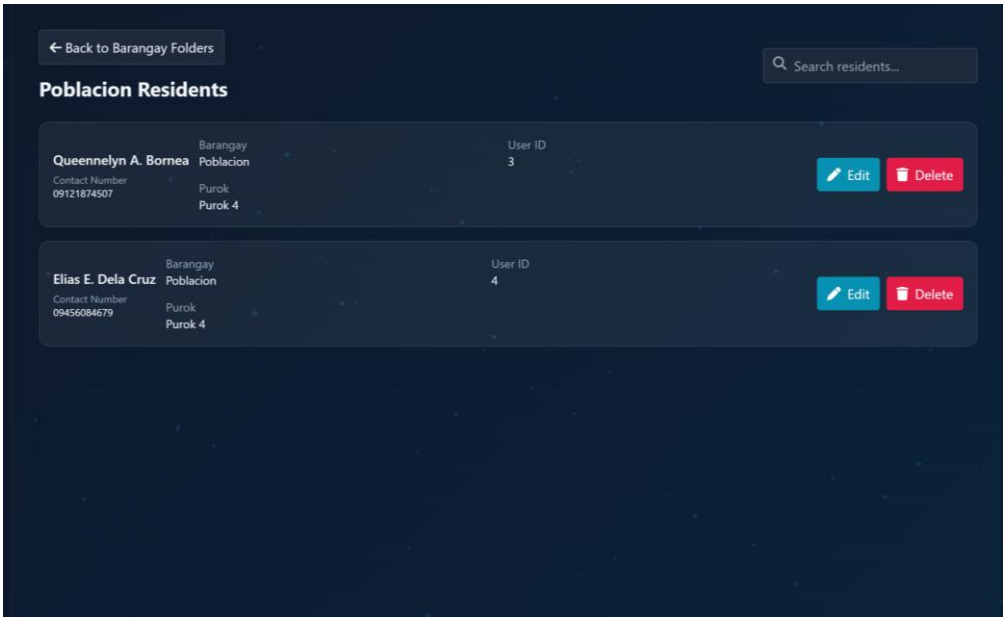


Image 2.1 Admin – Resident Management

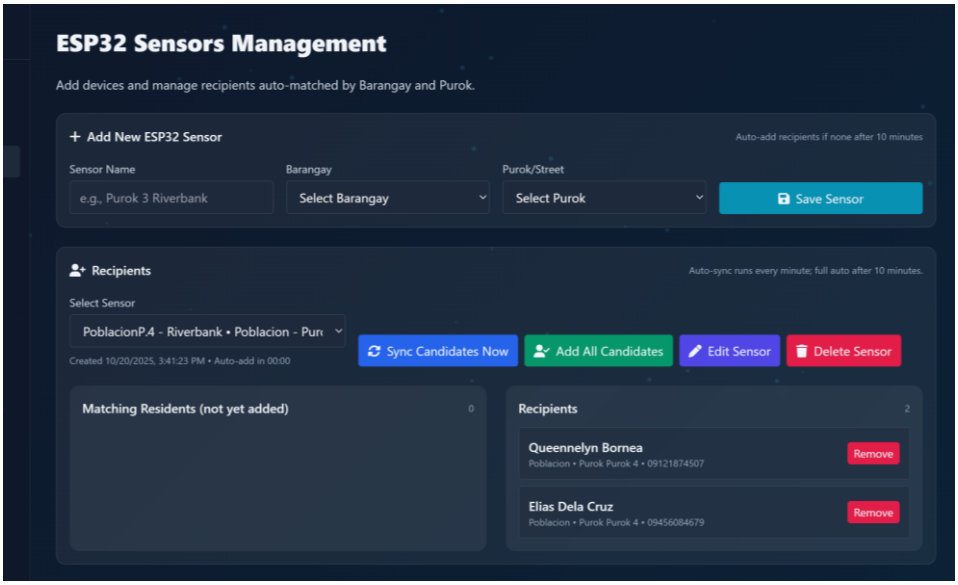


Image 2.2 Admin – ESP32 Sensors

The back-end interfaces include Resident Management (Image 2.1 Admin – Resident Management), which allows LDRRMO staff to manage user contact details and location, crucial for targeted alerts, and the ESP32 Sensors management page (Image 2.2 Admin – ESP32 Sensors), which facilitates the registration of new sensor units and the automatic linking of recipients by Barangay and Purok.

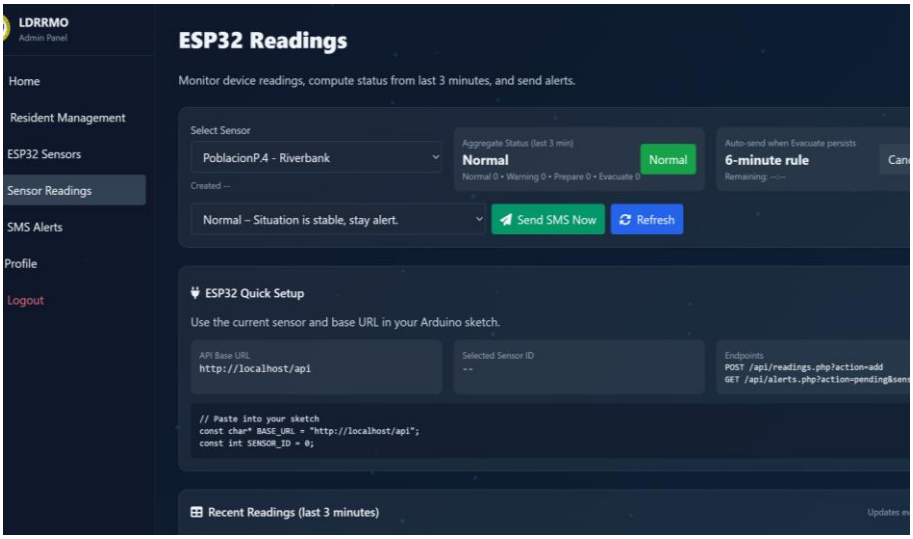


Image 2.3 Admin – Sensor Readings

The core operational interface is the Sensor Readings page (Image 2.3 Admin – Sensor Readings), where the administrator monitors real-time status and manages the automated alert process. This page displays the current status and the countdown for

the **6-minute rule**, which is the failsafe timer for automatic evacuation alerts. This functionality was a critical focus of the **Sensor Readings Response test (TC #: 003)**.

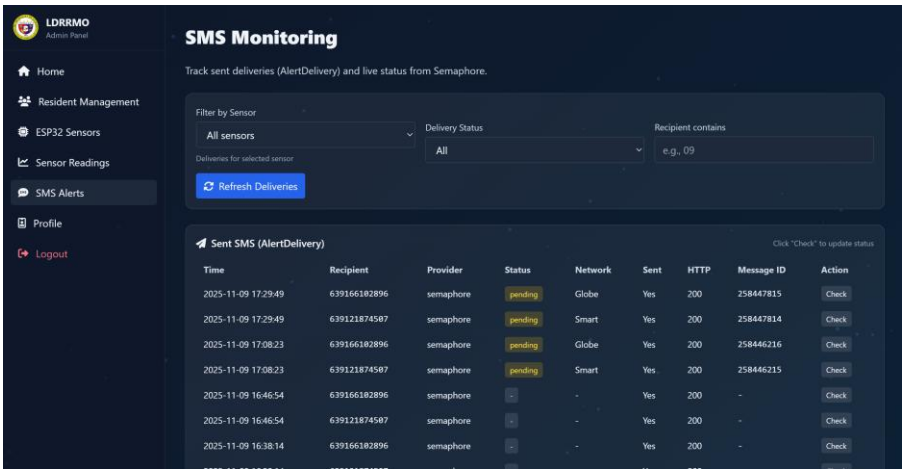


Image 2.4 Admin – SMS Alert

Furthermore, the system incorporates robust auditing via the SMS Monitoring screen (Image 2.4 Admin – SMS Alert Monitoring), which tracks the delivery status, provider, HTTP response codes, and network for every single outgoing message, ensuring complete traceability and accountability.

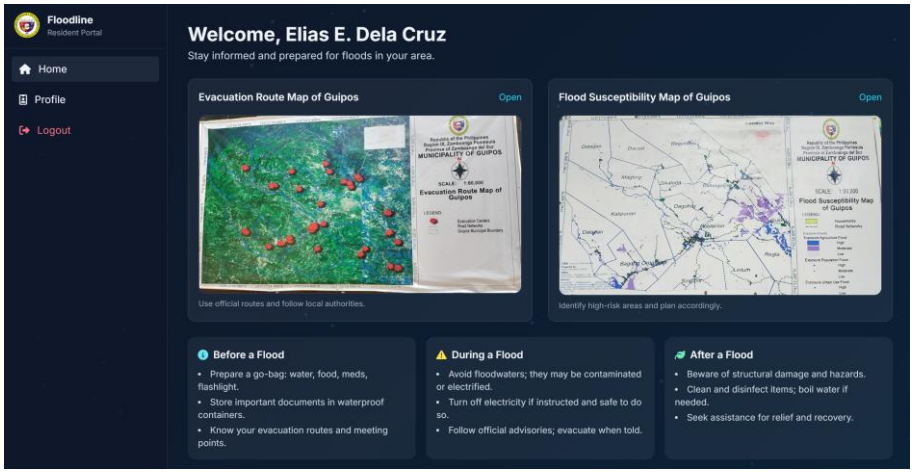


Image 3.0 Resident – Homepage

The platform also features a dedicated Resident Portal (Image 3.0 Resident – Homepage) that integrates high-resolution mapping of the Municipality of Guipos. By visualizing evacuation centers and flood-prone zones alongside official safety guidelines, the system bridges the gap between complex geospatial data and everyday citizen safety, facilitating rapid decision-making during emergency scenarios.

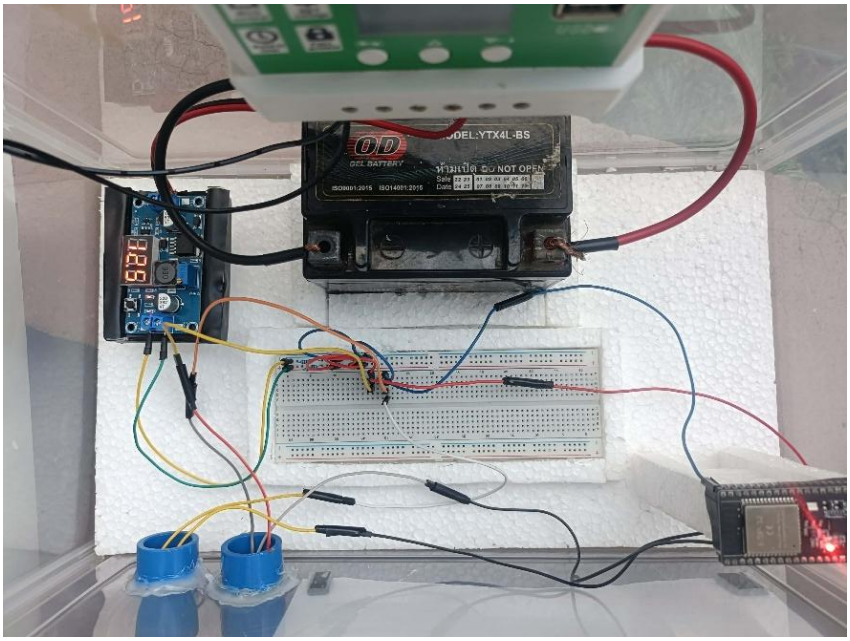


Image 4.0 Floodline Alert: ESP32 Prototype

The successful passing of **Data Logging / Output (TC #: 008)** confirms the integrity of this audit trail. The physical component, the Floodline Alert ESP32 Prototype (Image 4.0 Floodline: Alert: ESP32 Prototype), is the source of all data feeding this robust web environment.

Conclusion

Based on the applied evidence gathered from the eight test cases, the Floodline Alert System is concluded to be a stable, reliable, and functional flood warning solution that successfully achieved its general objective: to develop a reliable and cost-effective flood warning system that provides timely SMS alerts and enhances disaster preparedness for flood-prone communities in Guipos, Zamboanga del Sur.

The successful implementation confirms the fulfillment of all specific objectives:

1. To design an Arduino-based flood monitoring system integrated with water level sensor or float switch and GSM modules to detect rising floodwaters accurately. The system successfully integrated the ESP32 microcontroller with a dual-sensor unit (analog and float switch) and the backend API, confirming

its capability to detect rising floodwaters accurately, as validated by Accuracy and Stability Test (TC #: 002).

2. To develop an automated SMS notification feature that sends real-time flood alerts to registered users, ensuring accessibility even without internet connectivity. The system validated the integrity of its SMS dispatch via the HTTPSMS gateway, and the Sensor Readings Response test (TC #: 003) confirmed the failsafe 6-minute automated SMS dispatch, guaranteeing notification delivery.
3. To implement a solar-powered backup system using solar panels and power banks to maintain uninterrupted operation during power outages. While the provided tests focused on electrical power stability (Power Supply Check, TC #: 006), the overall system design accounts for continuous operation, demonstrated by its high stability during Stress Testing (TC #: 005).
4. To test the complete Floodline Alert system in selected barangays of Guipos to evaluate its accuracy, reliability, and user responsiveness. The comprehensive testing across eight test cases, including system integration (TC #: 004) and performance testing, confirmed the system's accuracy, reliability, and readiness for field deployment in the community.

The successful completion of Alpha Testing and Beta Testing fixed all major bugs and collected valuable user feedback, establishing a robust software foundation. The most critical finding relates to system resilience and safety: Stress Testing (TC #: 005) confirmed the system's ability to operate continuously for a whole day without crashing, supported by the back-end's efficiency in deleting older datasets from the ESP32 to prevent overload. Crucially, the system’s non-functional success provides confidence in its field deployment. Safety Testing (TC #: 007) confirmed that the device

handles abnormal conditions, such as sensor disconnections, by safely displaying errors without catastrophic failure or permanent damage.

The comprehensive logging capability (TC #: 008) ensures that the LDRRMO maintains a complete and auditable record of all sensor readings and every SMS delivery attempt, fulfilling the requirements for accountability and post-event analysis. The successful implementation and integration of the hardware and software components confirm the system is ready for effective deployment as a reliable tool for disaster risk reduction.

Recommendation

After the evaluation of the findings and successful implementation of the system, the following recommendations are provided for enhancement of its scope and functionality:

- For future iterations, it is recommended that the system be enhanced to integrate with external weather APIs (e.g., PAGASA) to receive forecast rainfall data and enable proactive flood prediction capabilities.
- The development of a dedicated native mobile application for LDRRMO administrators is recommended to ensure faster, continuous access to critical monitoring data and the manual alert function, especially during emergencies.
- It is recommended that automated software routines for sensor self-checking be implemented to monitor the validity and calibration status of the ESP32 readings, thereby improving long-term accuracy and maintenance scheduling.
- It is recommended to implement a three-level float switch system (or three individual float switches positioned at different heights) as an alternative for expanded data gathering and significantly more accurate water level readings, enabling graduated flood alerts based on precise multi-point detection.

REFERENCES

Abalos, M. S., & Fajardo, A. C. (2023). IoT based flood detection, alarm and monitoring system using multilayer perceptron and regression. *International Journal of Multidisciplinary Research and Analysis, 6*(4), 30–40. *International Journal of Multidisciplinary Research and Publications, 5*(1), 23–29. (Accessed July 18, 2025)

Abella, A. P. N., & Enriquez, M. D. (2021). Community-based flood alert system using LoRa/GSM. Mindoro Journal of Social Sciences and Development Studies. https://journal.omsc.edu.ph/index.php/mjssds/article/view/10 | View of Community-based flood alert system using long-range technology for Brgy. San Agustin, San Jose, Occidental Mindoro (Accessed July 18, 2025)

EPAM Startups & SMBs. (2024). Software Development Models Comparison (SDLC). https://startups.epam.com/blog/software-development-models-comparison (Accessed July 25, 2025)

Float. (2025). Agile vs. Waterfall: 10 key differences between the two methods. https://www.float.com/resources/agile-vs-waterfall (Accessed July 25, 2025)

Forbes Advisor. (2024). Agile vs. Waterfall: Which project management methodology is best? https://www.forbes.com/advisor/business/agile-vs-waterfall-methodology/ (Accessed July 26, 2025)

GeeksforGeeks. (2025). Difference between RAD model and Waterfall model. https://www.geeksforgeeks.org/software-engineering/difference-between-rad-model-and-waterfall-model/ (Accessed July 25, 2025)

Kamau, G. (2023). A real-time flood monitoring and alert system: Enhancing community resilience through technology integration – A case study of River Tana in Kenya. *The International Journal of Engineering and Science, 13*(12), 72-80. https://theijes.com/papers/vol13-issue12/13127280.pdf (Accessed July 20, 2025)

Nur Anis Athirah, N. H. Radzi, M. N. Abdullah, S. A. Jumaat, & N. Z. Mohamad. (2020). Solar-powered flood early warning system with short message service (SMS) notifications. *Indonesian Journal of Electrical Engineering and Computer Science, 18*(3), 1156-1162. https://doi.org/10.11591/ijeecs.v18.i3.pp1156-1162
Available at: https://www.researchgate.net/publication/341795715_Solar-powered_flood_early_warning_system_with_short_message_service_SMS_notifications (Accessed July 21, 2025)

Nurzaid Muhd Zain, Lidia Shahira Elias, Zulfikri Paidi, & Mahfudzah Othman. (2020).
Flood Warning and Monitoring System (FWMS) using GSM Technology. *Journal of Computing Research and Innovation (JCRINN), 5*(1), 7-15. https://www.researchgate.net/publication/350490232_Flood_Warning_and_Monitoring_System_FWMS_using_GSM_Technology (Accessed July 21, 2025)

Simplilearn. (2025). What is the Waterfall Model in Project Management? https://www.simplilearn.com/tutorials/project-management-tutorial/waterfall-model-in-project-management (Accessed July 26, 2025)

Tolentino, R. C., Reyes, M. L., & Santos, A. B. (2021). Real time flood detection, alarm
and monitoring system using rain gauge, float switch, flow rate meter sensor and Arduino microcontroller. *Philippine Journal of Science, 150*(S1), 101–113. (Accessed July 18, 2025)

APPENDICES

APPENDIX A

TEST CASES

The "Floodline Alert: Arduino-Based Flood Warning with SMS Notification" project underwent a comprehensive series of test cases to verify the integrity of its hardware and software components. Initial testing confirmed successful Sensor Initialization and dashboard connectivity, ensuring the ESP32 and water level sensors were fully operational before data collection began. The system demonstrated high accuracy during Sensor Data Reading tests, correctly identifying and displaying various water level thresholds including "Normal," "Warning," "Prepare," and "Evacuate" statuses.

Further evaluation focused on the system's responsiveness and long-term stability. The integration of the HTTPSMS gateway proved successful, with the system correctly transitioning from administrator-led commands to automatic SMS dispatch after a six-minute critical threshold was maintained. Additionally, the project passed rigorous Stress and Safety Tests, remaining operational for 24 hours without overheating and safely handling simulated faults such as sensor disconnections.

Final tests confirmed that all event data and alert statuses were accurately logged with timestamps, ensuring a reliable and retrievable history of flood activity.

Test Case #:	**001**	**Test Case Name:**	Sensor Initialization
Project Title:	Floodline Alert: Arduino-Based Flood Warning with SMS Notification		
Executed by:	Queennelyn A. Bornea		
Short Description: This test case used to test the initialization/activation of sensor components, source code variables (defaults, constants, global, local).			

Pre-condition: The prototype of the device should be supplied with enough power supply to run and initialize both software and hardware components.

Step	Method/Procedure	Expected Output	Actual Output	Status
1	Plug in device to power supply.	ESP32 light indicator should have blinking red light.	The ESP32 indicator light was observed blinking red.	Pass
2	Open Dashboard, initialize device lookup.	Available device/s will be connected to the dashboard.	The device successfully connected and appeared as 'Online' on the dashboard.	Pass
3	Open Sensor Readings.	Sensor Readings should be active.	The water level data field displayed “normal” indicating activity.	Pass

Post-Condition: Device will be able to collect alerts from water level sensor and float switch and deliver datasets to the database.

Test Case #:	**002**	**Test Case Name:**	Sensor Data Reading
Project Title:	Floodline Alert: Arduino-Based Flood Warning with SMS Notification		
Executed by:	Sharlene Nicole O. Navales		

Short Description: This test case checks accuracy and stability of real-time water level sensor and float switch

Pre-condition: Sensor must be initialized.

Step	Method/Procedure	Expected Output	Actual Output	Status
1	Start real-time monitoring.	Displayed water level and alerts refresh continuously.	The water level display updated every 10 seconds confirming continuous monitoring.	Pass
2	Generate “normal” (cm <= 30.0f) water level near sensor.	Displayed sensor readings shows normal status.	The sensor reading stabilized at 30 cm. and the dashboard displayed the corresponding "Normal" status.	Pass
3	Generate “warning” (cm <= 50.0f) water level near sensor.	Displayed sensor readings shows warning status.	The sensor reading stabilized at 50 cm. and the dashboard displayed the "Warning" status.	Pass
4	Generate “prepare” (cm <= 70.0f) water level near sensor.	Displayed sensor readings shows prepare status.	The sensor reading stabilized at 70 cm. and the dashboard displayed the "Prepare" status.	Pass
5	Generate “evacuate” (cm > 70.0f) water level near sensor and the float switch is activated.	Displayed sensor readings shows evacuate status.	The water level reading exceeded 70 cm, and the float switch state changed to 'Active', and the dashboard displayed the critical "Evacuate" status.	Pass

Post-Condition: System begins stable and accurate data recording.

Test Case #:	**003**	**Test Case Name:**	Sensor Readings Response
Project Title:	Floodline Alert: Arduino-Based Flood Warning with SMS Notification		
Executed by:	Kent Peter M. Rondrique		
Short Description: This test evaluates the responsiveness of outputs (Alert).			
Pre-condition: Sensor must provide valid data.			

Step	Method/Procedure	Expected Output	Actual Output	Status
1	Water level above threshold (“Evacuate”).	The sensor readings are accurate, and the system is ready to send the SMS notification upon receiving a direct command from the administrator.	Dashboard showed "Evacuate" status, and the SMS prompt was ready for Admin input.	Pass
2	Maintain “Evacuate” status in a span of 6 minutes.	SMS notifications to residents must be sent automatically by the system, without requiring manual intervention or a direct command from the administrator.	System automatically dispatched the "Evacuate" SMS notification to residents.	Pass
3	Water level below higher thresholds (“Normal”, “Warning”, “Prepare”).	The administrator monitors the situation and determines which automated SMS alert status should be sent.	Dashboard displayed the lower alert status (e.g., "Warning") and waited for Admin decision.	Pass

Post-Condition: Sensor readings responded correctly to water level and float switch status.

Test Case #:	**003**	**Test Case Name:**	Integration: Sensor + HTTPSMS gateway
Project Title:	Floodline Alert: Arduino-Based Flood Warning with SMS Notification		
Executed by:	Kent Peter M. Rondrique		
Short Description: This test verifies the combined behavior of Sensors and the HTTPSMS under real conditions.			
Pre-condition: Both Sensors and HTTPSMS must be functional.			

Step	Method/Procedure	Expected Output	Actual Output	Status
1	Simulate real water level alert > threshold.	The system presents the determined alert statuses and awaits the operator's command to send the automated SMS notifications to residents.	System displayed critical status and waited for Admin command to send SMS.	Pass

2	Lower the status; ensure sensor returns to normal.	The system regularly collects and processes new data batches, automatically deleting old records to prevent overload. It then calculates the average of the highest water levels to determine the appropriate alert status for the administrator to send.	System collected new data, averaged the highest levels, and correctly displayed the lower status.	Pass
3	Repeat the water level twice.	System responds consistently in all attempts.	The system's response (alert status and prompt) was identical for both repetitions.	Pass
Post-Condition: System demonstrates proper integrated functionality.				

Test Case #:	**004**	**Test Case Name:**	Stress Test	
Project Title:	Floodline Alert: Arduino-Based Flood Warning with SMS Notification			
Executed by:	Kent Peter M. Rondrique			
Short Description: This test evaluates long-duration performance and stability.				
Pre-condition: Device should be fully operational.				
Step	**Method/Procedure**	**Expected Output**	**Actual Output**	**Status**
1	Run system continuously for Whole day	No system crashes or overheating.	System remained operational for 24 hours without crashing or signs of overheating.	Pass
2	Record water level status at 10 seconds intervals.	Values remain stable with slight variations.	Recorded values showed slight, expected variations (e.g., 0.5cm) but remained stable within the set alert thresholds.	Pass
3	Trigger multiple waler level statuses and alerts periodically.	The sensor data remains constant across multiple collection periods.	Status transitions and alerts were accurately triggered and consistently reported each time.	Pass
Post-Condition: System remains stable under continuous use.				

Test Case #:	**005**	**Test Case Name:**	Power Supply Check
Project Title:	Floodline Alert: Arduino-Based Flood Warning with SMS Notification		
Executed by:	Kent Peter M. Rondrique		
Short Description: This test ensures the device operates correctly under proper voltage levels.			

Pre-condition: Multimeter must be available.				
Step	**Method/Procedure**	**Expected Output**	**Actual Output**	**Status**
1	Measure Arduino 5V pin using multimeter.	Voltage reads between 4.8–5.1V.	Multimeter read 5.1V.	Pass
2	Verify device restarts properly after power interruption.	System reinitializes without error.	Device successfully powered off and restarted, reconnecting to the dashboard without error.	Pass
Post-Condition: Power supply remains stable and safe.				

Test Case #:	**007**	**Test Case Name:**	Safety Test	
Project Title:	Floodline Alert: Arduino-Based Flood Warning with SMS Notification			
Executed by:	Queennelyn A. Bornea			
Short Description: This test checks safety under abnormal electrical or sensor conditions.				
Pre-condition: Must follow safety precautions.				
Step	**Method/Procedure**	**Expected Output**	**Actual Output**	**Status**
1	Simulate sensor disconnection	System displays error or no reading.	Dashboard displayed "-" error message indicating sensor is disconnected.	Pass
2	Simulate short circuit on non-critical pin.	System shuts off or alerts without damage.	The system continued operation without damage or a crash.	Pass
3	Restart system after fault simulation.	System recovers successfully.	System restarted and reconnected successfully to the dashboard.	Pass
Post-Condition: System safely handles abnormal conditions.				

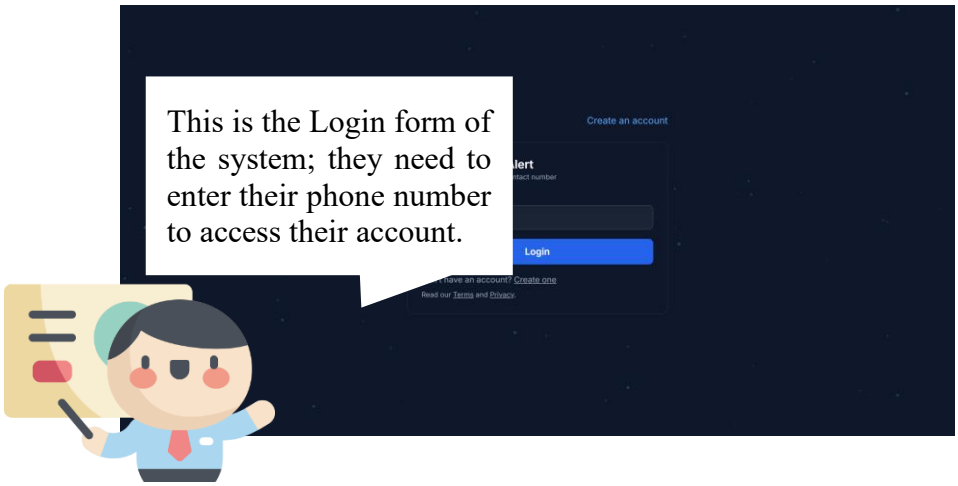
Test Case #:	**008**	**Test Case Name:**	Data Logging / Output	
Project Title:	Floodline Alert: Arduino-Based Flood Warning with SMS Notification			
Executed by:	Sharlene Nicole O. Navales			
Short Description: This test evaluates system's ability to record and store noise data.				
Pre-condition: Storage component must be functional.				
Step	**Method/Procedure**	**Expected Output**	**Actual Output**	**Status**

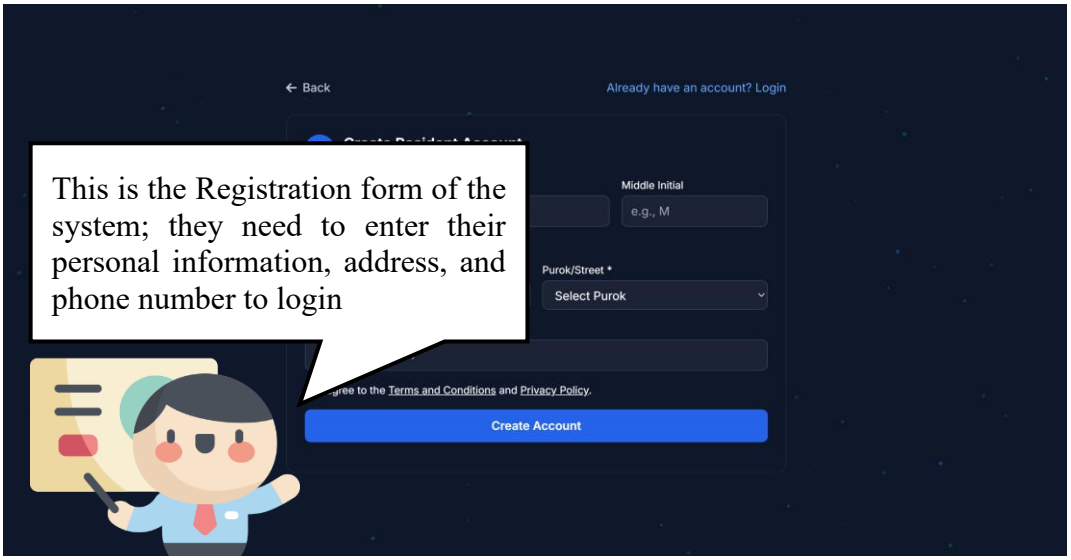
1	Trigger multiple water level and SMS alert statuses events.	Each event logged with timestamp.	All triggered events (water level, SMS status) were logged with accurate timestamps.	Pass
2	Open stored logs from SD/Serial	Data is readable and complete.	Log file opened successfully; all data was readable and complete.	Pass
3	Verify organization of log entries	Logs follow structured format.	Log entries followed the predefined structured format	Pass
Post-Condition: Noise data is stored, organized, and retrievable.				

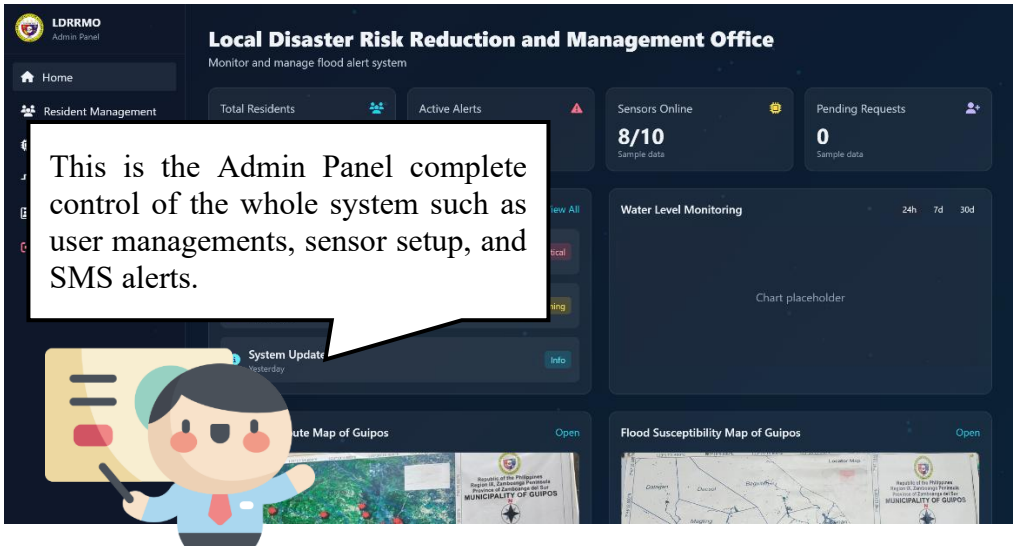
APPENDIX B

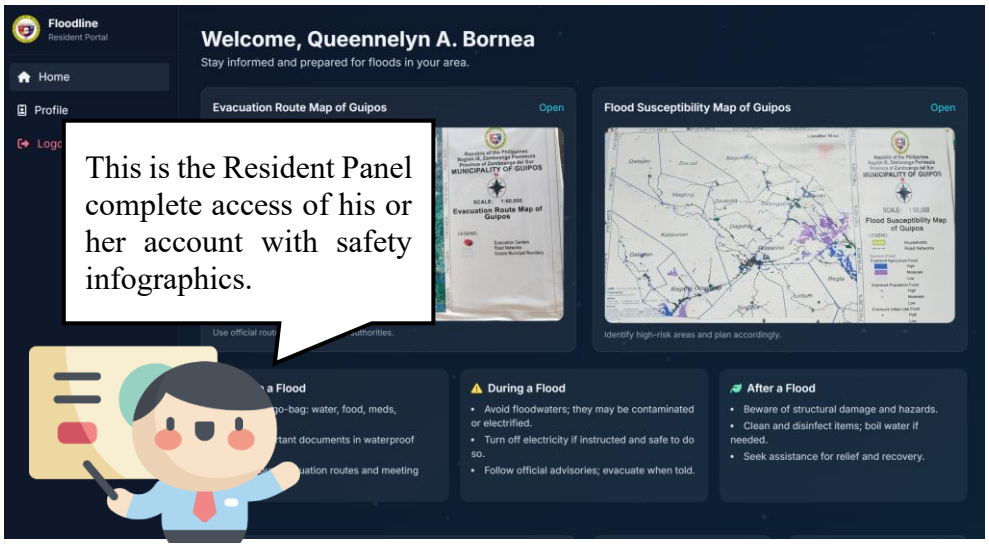
Story Board







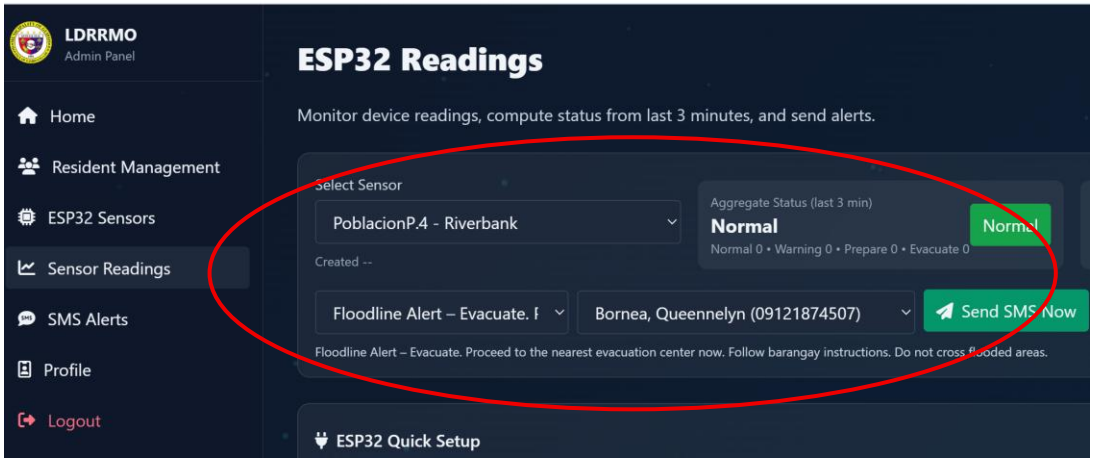

LDRRMO
Admin Panel
Home
Resident Management
Local Disaster Risk Reduction and Management Office
Monitor and manage flood alert system
Total Residents
Active Alerts
Sensors Online
8/10
Sample data
Pending Requests
0
Sample data
This is the Admin Panel complete control of the whole system such as user managements, sensor setup, and SMS alerts.
Water Level Monitoring
24h
7d
30d
Chart placeholder
System Update
Yesterday
Info
Open
Flood Susceptibility Map of Guipos
Open


Floodline
Resident Portal
Home
Profile
Welcome, Queennelyn A. Bornea
Stay informed and prepared for floods in your area.
Evacuation Route Map of Guipos
Open
Flood Susceptibility Map of Guipos
Open
This is the Resident Panel complete access of his or her account with safety infographics.
During a Flood
Avoid floodwaters; they may be contaminated or electrified.
Turn off electricity if instructed and safe to do so.
Follow official advisories; evacuate when told.
After a Flood
Beware of structural damage and hazards.
Clean and disinfect items; boil water if needed.
Seek assistance for relief and recovery.

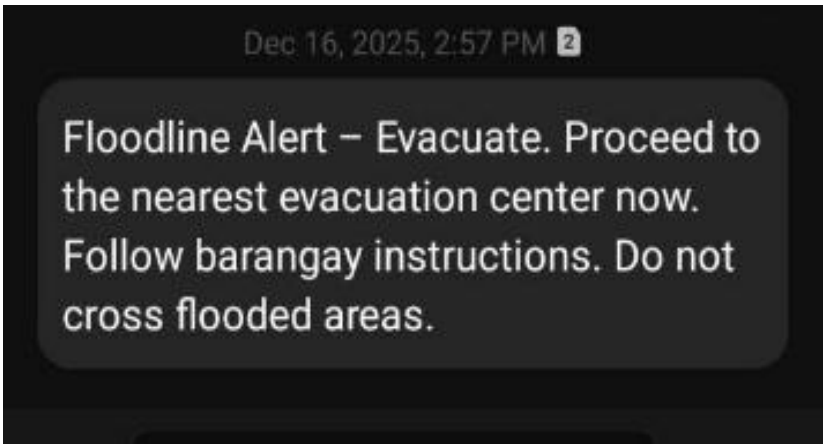
APPENDIX C

SAMPLE INPUT/OUTPUT

Admin sending SMS Alert to their respective residents



The image demonstrates how the administrator sends SMS alerts to residents within a specific sensor area. The admin can select the appropriate alert level and choose whether to notify all residents or a specific individual. In this example, the admin has selected a single user and set the alert status to 'Evacuate'.



The image above illustrates the SMS notification residents receive based on the alert level forwarded by the administrator. Additionally, if the time interval exceeds the six-minute mark, the system will automatically dispatch the SMS to all residents in the area.

APPENDIX D

PHOTO DOCUMENTATION

Interview



The image shows the reaserachers interviewed with Engineer Renan F. Mangle the MGDH of the LDRRMO of the Municipality of Guipos to gather insights.

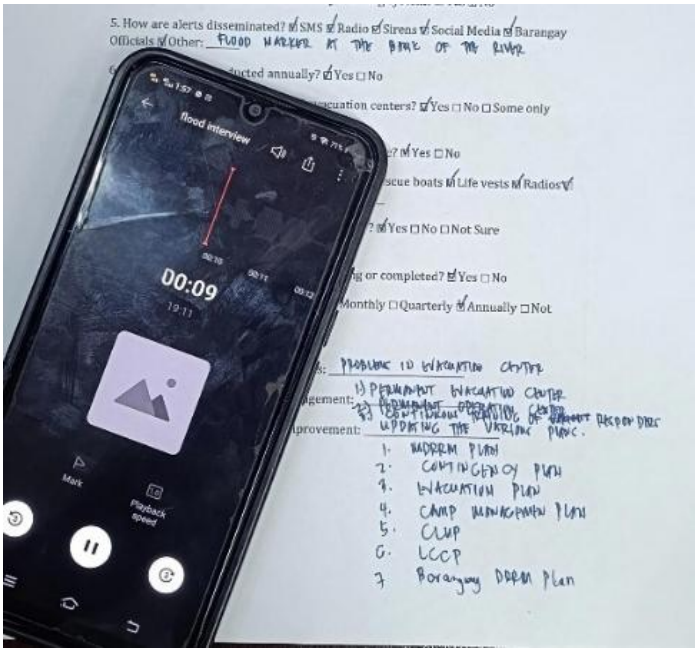


The image above shows one of the researchers conducted an interview and survey with one of the residents of the Municipality of Guipos in order to gather the necessary data to develop the system and provide solution.



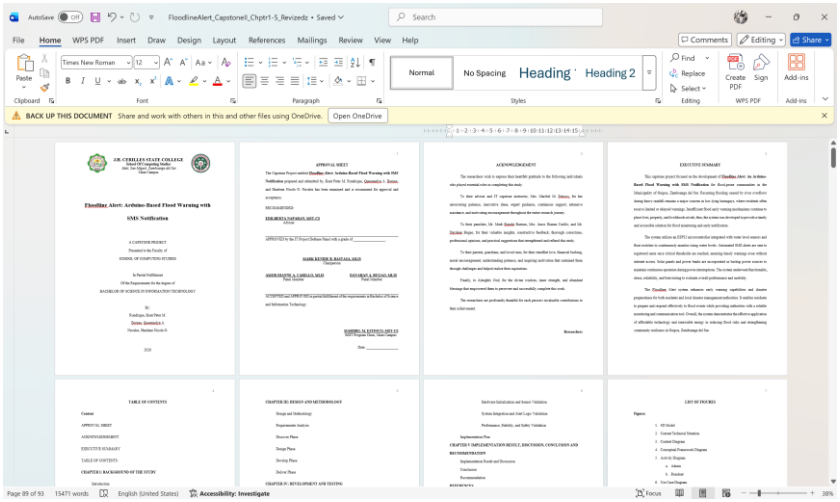
The image above depicts a resident being interviewed and surveyed regarding flood alerts to assess and gather relevant data for the study.

Audio Recording and Surveys



To ensure the system addresses real-world challenges, field interviews were conducted and recorded as shown in the image. The accompanying survey results provide empirical evidence regarding existing disaster management plans (e.g., MDRRM and Contingency Plans) and identified gaps in evacuation center management, which served as the foundational data for the proposed digital solution.

Making Document

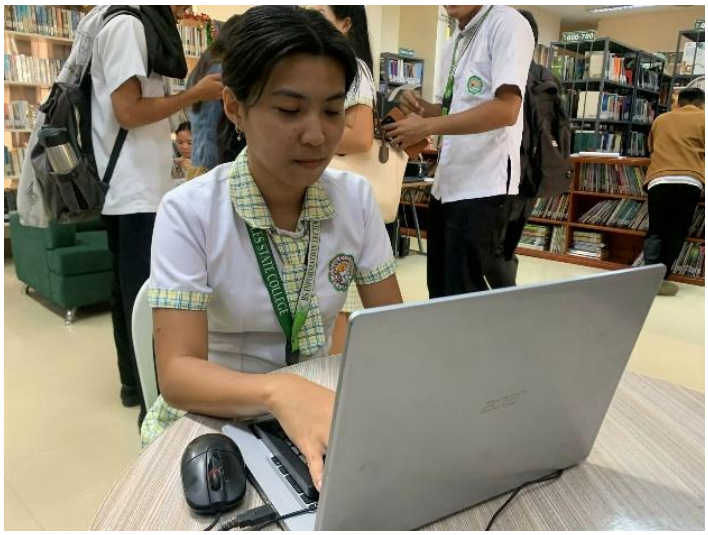


This screen capture presents the comprehensive capstone manuscript, titled 'Floodline Alert: Arduino-Based Flood Warning with SMS Notification.' It showcases the structured organization of the paper, ranging from the Executive Summary to the detailed Design and Methodology (Chapter III), serving as the formal academic record of the project's development, testing, and validation results.



The image captures the initial drafting of the system's architectural framework. Here, a researcher reviews technical flowcharts and logical diagrams within a collaborative academic environment. This stage was critical for aligning the system's

functional requirements with the established project goals before the start of the formal development phase.



This image depicts the synthesis and technical writing phase of the project. Researchers utilized university facilities to consolidate field data, hardware test results, and user feedback into the final document. This stage focused on ensuring that all technical specifications and implementation results were accurately recorded to maintain the study's academic integrity and clarity.

APPENDIX E

CURRICULUM VITAE



Personal Data

Full Name	Queennelyn A. Bornea
Gender	Female
Date of Birth	February 17, 2004
Place of Birth	Canunan, Guipos Zamboanga del Sur
Civil Status	Single
Citizenship	Filipino
Religion	Roman Catholic
Address	Purok Ipil-Ipil, Canunan, Guipos Zamboanga del Sur
Father's Name	Pascual A. Bornea
Occupation	Farmer
Present Address	Canunan, Guipos Zamboanga del Sur
Mother's Name	Ermelina A. Bornea
Occupation	Housewife
Present Address	Canunan, Guipos Zamboanga del Sur

Educational Background

Education	**School**	**Year Graduated**
Primary	Canunan Elementary School	2016
Secondary	Balongating National High School	2022
Tertiary	J.H. Cerilles State College	2026
Degree	Bachelor of Science in Information Technology	

CURRICULUM VITAE



Personal Data

Full Name	Sharlene Nicole O. Navales
Gender	Female
Date of Birth	June 16, 2004
Place of Birth	Sibucao, Dumalinao Zamboanga del Sur
Civil Status	Single
Citizenship	Filipino
Religion	Roman Catholic
Address	Purok Maisan 1, Sibucao, Dumalinao Zamboanga del Sur
Father's Name	Richard L. Navales
Occupation	Farmer
Present Address	Purok Maisan 1, Sibucao, Dumalinao Zamboanga del Sur
Mother's Name	Lenie O. Navales
Occupation	Teacher
Present Address	Purok Maisan 1, Sibucao, Dumalinao Zamboanga del Sur

Educational Background

Education	**School**	**Year Graduated**
Primary	Sibucao Elementary School	2016
Secondary	Southern Mindanao Colleges – San Fransico	2022
Tertiary	J.H. Cerilles State College	2026
Degree	Bachelor of Science in Information Technology	

CURRICULUM VITAE



Personal Data

Full Name	Kent Peter M. Rondrique
Gender	Male
Date of Birth	December 29, 2001
Place of Birth	Digon, Margosatubig Zamboanga del Sur
Civil Status	Single
Citizenship	Filipino
Religion	Roman Catholic
Address	Poblacion, Margosatubig Zamboanga del Sur
Father's Name	Cesar M. Rondrique
Occupation	Fisherman
Present Address	Poblacion, Margosatubig Zamboanga del Sur
Mother's Name	Elsa M. Rondrique
Occupation	Teacher
Present Address	Poblacion, Margosatubig Zamboanga del Sur

Educational Background

Education	**School**	**Year Graduated**
Primary	SPED Margosatubig Regional Pilot School	2014
Secondary	Toribio Minor National High School	2020
Tertiary	J.H. Cerilles State College	2026
Degree	Bachelor of Science in Information Technology	